MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

VREMERE ADRIAN, FAF-232

Report

Laboratory Work n.3.1

Sensors — Signal Acquisition and Conditioning

on Embedded Systems

Checked by:
Martîniuc Alexei, *univ. asist.*
FCIM, UTM

Chişinău – 2026

Contents

1 Domain Analysis **4**

1.1 Objective of the Laboratory Work 4

1.2 Problem Definition 4

1.3 Task Statement 5

1.4 Used Technologies 5

1.4.1 Analog-to-Digital Conversion (ADC) 5

1.4.2 OneWire Communication Protocol 6

1.4.3 I2C Communication Protocol 7

1.4.4 UART Serial Communication 7

1.4.5 FreeRTOS Real-Time Operating System 8

1.4.6 Hysteresis-Based Threshold Detection 8

1.4.7 Signal Conditioning Pipeline 9

1.5 Hardware Components 9

1.5.1 Arduino Mega 2560 9

1.5.2 NTC Thermistor (Analog Temperature Sensor) 10

1.5.3 DS18B20 Digital Temperature Sensor 11

1.5.4 LCD 1602 with I2C Interface 12

1.5.5 LEDs with Current-Limiting Resistors 12

1.5.6 Resistors 13

1.6 Software Components 13

1.6.1 PlatformIO 13

1.6.2 Wokwi Simulator 14

1.7 Case Study: Industrial Temperature Monitoring Systems 15

2 Design **16**

2.1 System Architecture Diagrams 16

2.1.1 Component-Level Architecture 16

2.1.2 Layered System Architecture 17

2.1.3 FreeRTOS Task Architecture 18

2.2 Block Diagrams 19

2.2.1 Main Application Flowchart 19

2.2.2 Sensor Acquisition Task Flowchart 19

2.2.3 Signal Conditioning Task Flowchart 20

2.2.4 Threshold Alert FSM State Diagram 21

2.2.5 Display Task Flowchart 22

2.3 Electrical Schematics 23

2.3.1 Component Specification 24

2.3.2 Circuit Connections 24

2.3.3 Hardware Configuration . . . 25
2.4 Project Structure . . . 25
2.5 Modular Implementation . . . 26
2.5.1 Driver Layer: AnalogTempSensor (NTC Thermistor) . . . 26
2.5.2 Driver Layer: DigitalTempSensor (DS18B20) . . . 28
2.5.3 Service Layer: ThresholdAlert Module . . . 30
2.5.4 Driver Layer: LcdDisplay (Reused) . . . 32
2.5.5 Driver Layer: Led (Reused) . . . 33
2.5.6 Driver Layer: StdioSerial (Reused) . . . 33
2.5.7 Service Layer: Shared Data and Synchronization . . . 34
2.5.8 Application Layer: Lab 3.1 Entry Point . . . 35

3 Obtained Results **36**
3.1 Build Process . . . 36
3.2 Simulation Startup . . . 36
3.3 Test Scenarios . . . 37
3.3.1 Test 1: Normal Operation . . . 37
3.3.2 Test 2: Analog Sensor Alert Trigger . . . 37
3.3.3 Test 3: Digital Sensor Alert Trigger . . . 38
3.3.4 Test 4: Dual Sensor Alert . . . 38
3.3.5 Test 5: Hysteresis Behavior . . . 38
3.3.6 Test 6: Debounce Validation . . . 39
3.4 Serial Monitor Output . . . 39
3.5 LCD Display Pages . . . 40
3.6 Verification Summary . . . 40

4 Conclusions **41**
4.1 Achievement of Objectives . . . 41
4.2 Performance Analysis . . . 41
4.3 Limitations . . . 42
4.4 Proposed Improvements . . . 42
4.5 Real-World Impact . . . 43

5 Note Regarding Usage of AI **43**

6 Bibliography **44**

7 Appendix — Source Code **45**
7.1 Driver Layer: AnalogTempSensor . . . 45
7.1.1 AnalogTempSensor.h . . . 45
7.1.2 AnalogTempSensor.cpp . . . 46
7.2 Driver Layer: DigitalTempSensor . . . 48

7.2.1 DigitalTempSensor.h 48
7.2.2 DigitalTempSensor.cpp 49
7.3 Service Layer: ThresholdAlert 51
7.3.1 ThresholdAlert.h 51
7.3.2 ThresholdAlert.cpp 52
7.4 Application Layer: Shared Data and Synchronization 55
7.4.1 sensor_data.h 55
7.4.2 sensor_data.cpp 57
7.5 Application Layer: Task 1 — Sensor Acquisition 57
7.5.1 task_acquisition.h 57
7.5.2 task_acquisition.cpp 58
7.6 Application Layer: Task 2 — Signal Conditioning 60
7.6.1 task_conditioning.h 60
7.6.2 task_conditioning.cpp 60
7.7 Application Layer: Task 3 — Display and Reporting 63
7.7.1 task_display.h 63
7.7.2 task_display.cpp 64
7.8 Application Layer: Lab 3.1 Entry Point 66
7.8.1 lab3_1_main.h 66
7.8.2 lab3_1_main.cpp 67

1. Domain Analysis

1.1. Objective of the Laboratory Work

The objective of this laboratory work is to design, implement, and validate a dual-sensor temperature monitoring system on an Arduino Mega 2560 microcontroller. The system reads temperature data from two distinct sensor types — an analog NTC thermistor and a digital DS18B20 — and applies signal conditioning techniques to detect threshold crossings reliably. The application demonstrates the fundamental principles of sensor acquisition, analog-to-digital conversion, digital communication protocols, hysteresis-based threshold detection, and software debounce filtering in the context of embedded systems.

This laboratory work addresses several critical aspects of embedded sensor systems. First, it explores the differences between analog and digital temperature sensing, highlighting the trade-offs in accuracy, conversion time, interface complexity, and noise susceptibility. Second, it implements a complete signal processing pipeline — from raw sensor data acquisition through conditioning and threshold evaluation to user-facing display and reporting. Third, the system uses FreeRTOS preemptive multitasking to separate the acquisition, conditioning, and display responsibilities into independent tasks with well-defined synchronization boundaries, demonstrating how real-time operating systems enable modular, responsive embedded applications.

The key learning objectives include understanding analog-to-digital conversion principles and the Steinhart-Hart equation for thermistor-based temperature measurement, mastering the OneWire digital communication protocol used by the DS18B20 sensor family, implementing hysteresis-based threshold detection to prevent alert oscillation near boundary values, applying software debounce filtering to validate the persistence of threshold crossings before triggering alerts, using FreeRTOS tasks with mutexes and binary semaphores for safe inter-task communication, and presenting processed sensor data through both an LCD display and a structured serial terminal interface.

1.2. Problem Definition

The task requires the development of a microcontroller-based temperature monitoring application with the following functional requirements. The system must periodically acquire temperature data from two sensors: one analog sensor (NTC thermistor) read via the built-in ADC, and one digital sensor (DS18B20) read via the OneWire protocol. The acquisition rate must be configurable, with a default period of 50 milliseconds to ensure responsive detection of temperature changes.

The acquired raw data must undergo signal conditioning through a threshold alerting module. This module implements hysteresis between two configurable values (for example, 28°C and 30°C) to prevent oscillation of the alert signal when the temperature hovers near the threshold boundary. Additionally, the module applies software debounce filtering that requires a configurable number of consecutive threshold crossings (for example, 5 readings) before confirming a

state transition, ensuring that transient sensor noise or brief temperature spikes do not trigger false alerts.

The system must provide two output channels for displaying the monitoring results. An I2C LCD display shows the current temperature readings from both sensors and the alert status in a compact, human-readable format that alternates between information pages. Simultaneously, a structured report is printed to the serial terminal via STDIO (printf) at a lower frequency (every 2 seconds), providing detailed diagnostic information including raw ADC values, calculated resistance, converted temperatures, alert FSM states, debounce counters, threshold configuration, and cumulative statistics.

The application architecture must follow a modular design with clearly separated concerns. Sensor drivers, the threshold alert module, the LCD display driver, and the STDIO serial interface must each reside in reusable library modules under the PlatformIO project structure, while the lab-specific task orchestration resides in dedicated task files. The FreeRTOS scheduler manages three preemptive tasks: sensor acquisition (highest priority), signal conditioning with alerting (medium priority), and display with reporting (lowest priority).

Three LEDs provide immediate visual feedback on the system state: a green LED indicates normal operation (both sensors below threshold), a red LED lights when the analog sensor triggers an alert, and a yellow LED lights when the digital sensor triggers an alert. During debounce transitions, the corresponding LED blinks to indicate that a state change is being validated.

1.3. Task Statement

Sarcina de laborator — Achiziție semnal binar

Pentru un senzor analogic sau digital de temperatură, să se realizeze o aplicație pentru MCU care va achiziționa periodic semnalul de la senzorul selectat dintr-o listă de senzori, va afișa o alertă la depășirea unui prag prestabilit, afișând stările intermediare de condiționare și starea curentă în interfața STDIO, fie pe un display LCD, fie pe interfața serială.

Varianta de realizare: implementarea unui sistem de monitorizare a doi senzori, unul analogic și unul digital, cu afișarea valorilor și alertelor de la ambii senzori pe display LCD.

1.4. Used Technologies

Analog-to-Digital Conversion (ADC)

Analog-to-Digital Conversion is the process of translating a continuous analog voltage signal into a discrete digital representation that a microcontroller can process. The Arduino Mega 2560 features a 10-bit successive approximation ADC with 16 multiplexed input channels. The 10-bit resolution means the ADC maps the input voltage range (0 V to 5 V by default) into 1024 discrete levels, yielding a resolution of approximately 4.88 mV per step.

The ADC conversion process begins when the firmware triggers a conversion on the selected channel. The successive approximation register (SAR) algorithm performs a binary search through the possible output codes, comparing the input voltage against an internally generated reference at each step. On the ATmega2560, the ADC clock is derived from the system clock through a prescaler, and a full conversion requires 13 ADC clock cycles (25 cycles for the first conversion after enabling the ADC). At the default prescaler setting of 128 (yielding a 125 kHz ADC clock from the 16 MHz system clock), a single conversion takes approximately 104 microseconds.

For temperature measurement with an NTC thermistor, the ADC reading represents the voltage at the midpoint of a resistive voltage divider formed by the thermistor and a fixed series resistor. The raw ADC value is then converted to resistance using the voltage divider equation, and subsequently to temperature using the Steinhart-Hart Beta parameter equation. This two-step conversion process introduces quantization noise from the ADC and component tolerance errors from the resistors, both of which must be considered when evaluating measurement accuracy.

The Steinhart-Hart Beta parameter equation used for NTC-to-temperature conversion is:

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{B} \cdot \ln\left(\frac{R_{NTC}}{R_0}\right)$$

where T is the absolute temperature in Kelvin, T_0 is the reference temperature (typically 298.15 K = 25°C), B is the Beta coefficient of the thermistor (3950 K for the sensor used in this lab), R_{NTC} is the measured thermistor resistance, and R_0 is the nominal resistance at T_0 (10 kΩ). The resistance is calculated from the ADC reading using:

$$R_{NTC} = R_{series} \cdot \frac{ADC}{ADC_{max} - ADC}$$

where R_{series} is the fixed series resistor (10 kΩ), ADC is the raw digital reading, and ADC_{max} is the maximum ADC value (1023 for 10-bit resolution).

OneWire Communication Protocol

OneWire is a proprietary communication protocol developed by Dallas Semiconductor (now Maxim Integrated / Analog Devices) that enables bidirectional data exchange between a master device and one or more slave devices over a single data wire plus ground. The protocol is particularly significant in embedded temperature sensing because it allows the DS18B20 digital temperature sensor to communicate its readings using only one GPIO pin, minimizing the wiring complexity of multi-sensor installations.

The OneWire protocol operates at TTL voltage levels (typically 3.3 V or 5 V) and uses an open-drain bus topology with a pull-up resistor (typically 4.7 kΩ) to VCC. Communication is initiated exclusively by the master, which drives the bus low to generate timing slots. The protocol defines several fundamental operations: Reset/Presence pulse (the master drives the bus low for at least 480 μs, then releases it; slave devices respond by pulling the bus low for 60–240 μs), Write Time Slot (the master drives the bus low and either releases it quickly for a logical '1' or holds it low for a logical '0'), and Read Time Slot (the master briefly drives the bus low, then samples the

bus state; the slave holds the bus low for a ‘0’ or releases it for a ‘1’).

Each slave device on the OneWire bus has a unique 64-bit ROM code that serves as its address. The master can either address a specific device using the Match ROM command or communicate with all devices simultaneously using the Skip ROM command (useful when only one device is present). For temperature measurement, the master issues a Convert T command (0x44), which instructs the DS18B20 to begin its internal analog-to-digital conversion. The conversion time depends on the configured resolution: 93.75 ms at 9-bit, 187.5 ms at 10-bit, 375 ms at 11-bit, or 750 ms at 12-bit resolution. After conversion completes, the master reads the scratchpad memory containing the temperature result.

I2C Communication Protocol

Inter-Integrated Circuit (I2C) is a synchronous, multi-master, multi-slave serial communication protocol that uses two bidirectional open-drain lines: Serial Data (SDA) and Serial Clock (SCL). Developed by Philips Semiconductor (now NXP) in 1982, I2C has become one of the most widely used communication interfaces in embedded systems for connecting low-speed peripheral devices such as sensors, EEPROMs, real-time clocks, and LCD controllers.

[Image pending — save to resources/DomainAnalysis/UsedTechnologies/i2c_protocol.png]

***Figure 1.1** – I2C communication protocol timing diagram*

In this laboratory, I2C is used to communicate with the LCD 1602 display module through a PCF8574 I/O expander chip. The LCD module is configured at I2C address 0x27, and communication occurs at the standard 100 kHz clock speed. The Arduino Mega 2560 provides dedicated I2C pins: SDA on pin 20 and SCL on pin 21, both with internal pull-up capability. The I2C bus allows multiple devices to share the same two wires, making it possible to add additional sensors or displays without consuming additional GPIO pins.

Each I2C transaction follows a defined sequence: the master generates a START condition (SDA transitions from high to low while SCL is high), transmits the 7-bit slave address followed by a read/write bit, waits for an acknowledgment (ACK) from the addressed slave, transfers one or more data bytes (each followed by an ACK/NACK), and finally generates a STOP condition (SDA transitions from low to high while SCL is high). The LiquidCrystal_I2C library abstracts these low-level protocol details, providing a simple API for text display operations.

UART Serial Communication

UART (Universal Asynchronous Receiver-Transmitter) is a hardware communication protocol for asynchronous serial data exchange between two devices. In this laboratory, UART is used to establish the STDIO communication channel between the Arduino Mega 2560 and the host PC, enabling structured diagnostic reports to be displayed in a terminal emulator.

The Arduino Mega 2560 provides four independent hardware UART ports (Serial, Serial1, Serial2, Serial3). UART0 (Serial) is connected to the on-board USB-to-serial converter, enabling seamless communication with the host PC at the configured baud rate of 9600 bits per second. The STDIO library redirection mechanism (implemented in the StdioSerial library module) maps the standard C I/O functions — printf() for formatted output and fgets() for line input — onto the UART hardware, allowing the application to use familiar C library functions instead of Arduino-specific Serial.print() calls.

FreeRTOS Real-Time Operating System

FreeRTOS is a real-time operating system (RTOS) kernel for embedded microcontrollers that provides preemptive multitasking, inter-task synchronization primitives, and deterministic scheduling. In this laboratory, FreeRTOS manages three concurrent tasks that together implement the sensor monitoring pipeline: acquisition, conditioning, and display/reporting.

The FreeRTOS scheduler uses a priority-based preemptive algorithm: at any given moment, the highest-priority task that is ready to run receives CPU time. When a higher-priority task becomes ready (for example, when a semaphore it was waiting on is given), the scheduler immediately preempts the running lower-priority task and switches context to the higher-priority one. This mechanism ensures that time-critical operations (such as sensor acquisition) are never delayed by lower-priority work (such as LCD display updates).

The synchronization primitives used in this lab include binary semaphores and mutexes. A binary semaphore acts as a lightweight signaling mechanism: the acquisition task *gives* the semaphore after completing a reading cycle, and the conditioning task *takes* the semaphore to wake up and process the new data. This event-driven approach eliminates the need for polling and minimizes CPU usage. A mutex (mutual exclusion semaphore) protects the shared data structures accessed by multiple tasks, ensuring that only one task can read or modify the shared state at any time. FreeRTOS mutexes provide priority inheritance: if a low-priority task holds the mutex and a high-priority task attempts to acquire it, the low-priority task temporarily inherits the higher priority, preventing the pathological condition known as priority inversion.

Task timing in FreeRTOS is managed through two mechanisms. The `vTaskDelayUntil()` function provides drift-free periodic execution by suspending the task until an absolute tick count is reached, compensating for any jitter in the task's execution time. The `xSemaphoreTake()` function with `portMAX_DELAY` blocks the task indefinitely until the semaphore becomes available, providing efficient event-driven wake-up with zero polling overhead.

Hysteresis-Based Threshold Detection

Hysteresis is a signal conditioning technique that uses two distinct thresholds — an upper (high) threshold and a lower (low) threshold — to determine state transitions. Unlike single-threshold detection, which can produce rapid oscillation (chattering) when the signal hovers near the threshold value, hysteresis introduces a dead zone between the two thresholds where the system maintains its current state regardless of the input value.

In the context of temperature monitoring, hysteresis works as follows: the alert state activates when the temperature rises above the high threshold (e.g., 30°C) and deactivates only when the temperature drops below the low threshold (e.g., 28°C). If the temperature fluctuates between 28°C and 30°C, the alert state does not change — it remains in whatever state it was previously in. This 2°C hysteresis band prevents the annoying and potentially harmful scenario where an alert repeatedly activates and deactivates due to minor temperature fluctuations around a single threshold.

The ThresholdAlert module in this lab extends basic hysteresis with software debounce filtering. Even with hysteresis, a sudden transient spike (caused by electrical noise, sensor glitches, or external interference) could momentarily push the reading above the threshold. The debounce mechanism requires N consecutive readings (configurable, default 5) to confirm that the threshold has been genuinely crossed before committing to a state transition. If the reading drops back before N confirmations are reached, the transition is cancelled and the system returns to its previous state. This two-layer protection (hysteresis + debounce) provides robust and reliable threshold detection suitable for industrial and safety-critical applications.

Signal Conditioning Pipeline

Signal conditioning in embedded sensor systems refers to the complete chain of operations applied to raw sensor data before it is presented to the user or used for decision-making. In this laboratory, the signal conditioning pipeline consists of several stages, each implemented as a distinct software module.

The first stage is raw data acquisition, where the ADC samples the NTC thermistor voltage and the OneWire interface reads the DS18B20 digital output. The second stage is physical quantity conversion, where the raw ADC value is transformed into a temperature reading through the Steinhart-Hart equation (for the analog sensor) or directly reported in degrees Celsius (for the digital sensor, which performs its own internal conversion). The third stage is threshold evaluation with hysteresis and debounce, where the converted temperature is compared against configurable thresholds to determine the alert state. The fourth and final stage is output formatting, where the processed data is presented on the LCD display and serial terminal in a human-readable format.

Each stage operates at its own temporal resolution: acquisition runs at 50 ms (20 Hz), conditioning is event-driven (triggered by each acquisition), and display/reporting runs at 500 ms (2 Hz) for the LCD and 2 seconds (0.5 Hz) for the STDIO report. This multi-rate architecture balances responsiveness (fast sensor reading) with resource efficiency (reduced LCD refresh and serial output overhead).

1.5. Hardware Components

Arduino Mega 2560

The Arduino Mega 2560 is the central microcontroller development board used in this laboratory. It is based on the ATmega2560 AVR microcontroller, which features a 16 MHz clock

frequency, 256 KB of Flash program memory, 8 KB of SRAM, and 4 KB of EEPROM. The board provides 54 digital I/O pins (15 of which support PWM output), 16 analog input pins with 10-bit ADC resolution, 4 hardware UART serial ports, an SPI interface, and an I2C interface.

For this laboratory, the Arduino Mega 2560 serves multiple roles: it reads the analog NTC thermistor through its ADC (pin A0), communicates with the DS18B20 via a digital GPIO pin (pin 2) using the OneWire protocol, drives an I2C LCD display through its dedicated SDA/SCL pins (pins 20/21), controls three indicator LEDs through digital GPIO pins (pins 8, 9, 10), and runs the FreeRTOS scheduler to manage concurrent tasks. The board's generous memory (8 KB SRAM) is essential for running FreeRTOS with multiple tasks, each requiring its own stack allocation. The 256 KB Flash memory accommodates the application code, FreeRTOS kernel, sensor libraries, and the LCD display library without space constraints.

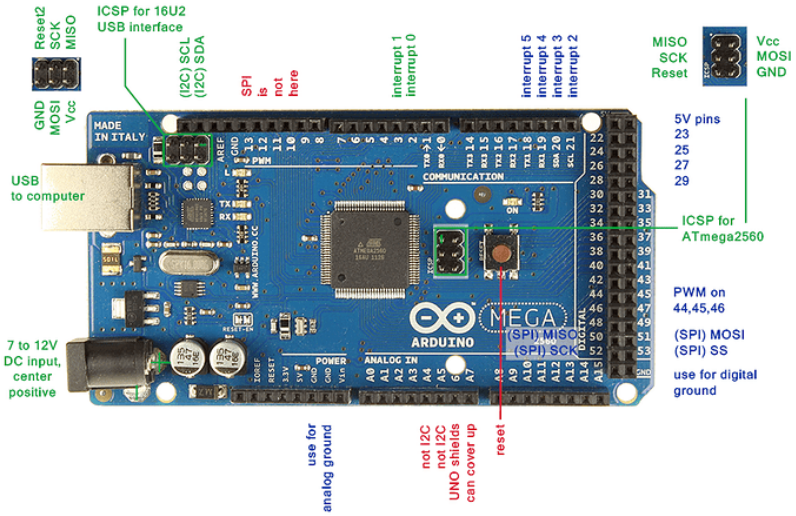

Figure 1.2 *– Arduino Mega 2560 development board*

NTC Thermistor (Analog Temperature Sensor)

An NTC (Negative Temperature Coefficient) thermistor is a resistive temperature sensor whose electrical resistance decreases as temperature increases. The thermistor used in this laboratory has a nominal resistance of 10 kΩ at 25°C and a Beta coefficient (B value) of 3950 K. These parameters define the resistance-temperature characteristic curve through the Steinhart-Hart Beta equation.

The NTC thermistor is connected in a voltage divider configuration with a 10 kΩ fixed series resistor. The midpoint voltage of this divider is fed to the Arduino's analog input pin A0. As the temperature changes, the thermistor resistance changes, altering the voltage division ratio and producing a corresponding change in the ADC reading. At 25°C, the thermistor resistance equals the series resistance (both 10 kΩ), placing the divider midpoint at exactly VCC/2 = 2.5 V, which corresponds to an ADC reading of approximately 512 out of 1023. At higher temperatures, the thermistor resistance decreases, pulling the midpoint voltage lower and producing a smaller ADC reading. At lower temperatures, the resistance increases, pushing the voltage higher.

The NTC thermistor's advantages include its high sensitivity (large resistance change per

degree), low cost, small physical size, and fast thermal response time. Its primary limitation is the nonlinear resistance-temperature relationship, which requires the Steinhart-Hart equation (or a lookup table) for accurate conversion. The 10-bit ADC resolution of the Arduino provides a theoretical temperature resolution of approximately 0.1°C near the 25°C nominal point, though practical accuracy is limited by component tolerances and electrical noise.

[Photo pending — save to resources/DomainAnalysis/Hardware-Components/ntc_thermistor.png]

***Figure 1.3** – NTC thermistor (10K, Beta=3950)*

DS18B20 Digital Temperature Sensor

The DS18B20 is a digital temperature sensor manufactured by Maxim Integrated (formerly Dallas Semiconductor) that provides 9-to-12-bit temperature measurements over a single-wire digital interface. Unlike the NTC thermistor, which outputs an analog voltage that must be externally converted, the DS18B20 performs its own internal analog-to-digital conversion and communicates the result digitally, eliminating analog noise and conversion errors in the communication path.

The DS18B20 measures temperatures from −55°C to +125°C with a configurable resolution. At 10-bit resolution (used in this lab), the sensor provides 0.25°C precision with a conversion time of approximately 187.5 ms. The sensor's accuracy is ±0.5°C over the −10°C to +85°C range, making it suitable for most environmental monitoring applications.

Communication with the DS18B20 uses the OneWire protocol over a single data pin (DQ), plus power (VCC) and ground (GND). A 4.7 kΩ pull-up resistor on the DQ line is required because the OneWire bus uses an open-drain topology. Each DS18B20 has a factory-programmed unique 64-bit serial code that enables multiple sensors to share the same OneWire bus and be individually addressed. In this laboratory, only one DS18B20 is used, and the Skip ROM command is employed for simplified single-device communication.

The DS18B20's key advantages over the NTC thermistor include digital output (immune to analog noise on the communication path), factory-calibrated accuracy, direct temperature output without external conversion circuitry, addressable multi-sensor capability, and user-configurable resolution. However, it requires a more complex communication protocol (OneWire), has a longer conversion time compared to an ADC reading, and costs more than a simple thermistor.

[Photo pending — save to resources/DomainAnalysis/Hardware-Components/ds18b20.png]

***Figure 1.4** – DS18B20 digital temperature sensor (TO-92 package)*

LCD 1602 with I2C Interface

The LCD 1602 is a character liquid crystal display module capable of showing 16 characters per line across 2 lines. When equipped with a PCF8574-based I2C backpack module, the display communicates with the microcontroller over just two wires (SDA and SCL) instead of requiring 6–10 parallel data pins. The I2C address is typically 0x27, though some modules use 0x3F depending on the address jumper configuration.

In this laboratory, the LCD serves as the primary real-time display for temperature readings and alert status. The display alternates between two information pages every 2 seconds: the first page shows the current analog and digital temperatures along with the overall system status (OK, ALERT: Analog, ALERT: Digital, or ALERT: A+D), while the second page displays the configured threshold values for reference. The 16-character line width constrains the information density, requiring careful formatting to present the most relevant data in a compact form.

Figure 1.5 *– LCD 1602 display with I2C backpack module*

LEDs with Current-Limiting Resistors

Three LEDs provide immediate visual feedback on the system state. Each LED is connected to a digital GPIO pin through a 220 Ω current-limiting resistor, which restricts the forward current to approximately 15 mA at the 5 V logic level (accounting for the typical 2 V forward voltage drop across the LED). The three colors encode the following system states:

The green LED (pin 8) indicates normal system operation, meaning both sensors are reading below their respective thresholds. The red LED (pin 9) indicates that the analog sensor (NTC thermistor) has triggered a threshold alert. The yellow LED (pin 10) indicates that the digital sensor (DS18B20) has triggered a threshold alert. During debounce transitions (when the system is counting consecutive confirmations), the corresponding LED blinks to provide a visual cue that a state change is being validated but not yet confirmed. When any alert is active, the green LED turns off to provide unambiguous status indication.

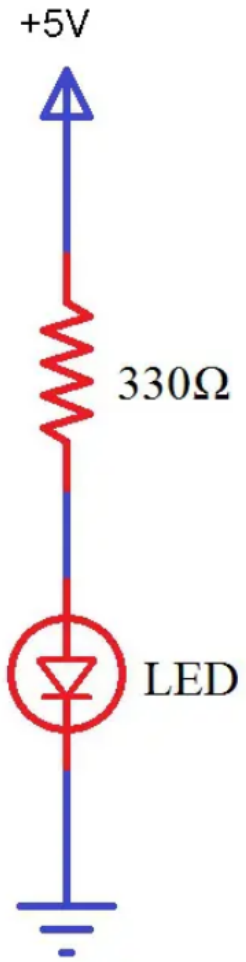

Figure 1.6 *– Standard 5mm LEDs (green, red, yellow)*

Resistors

Several resistors are used in the circuit. A 10 kΩ resistor forms the upper half of the voltage divider for the NTC thermistor, creating a known reference against which the thermistor's variable resistance produces a measurable voltage. A 4.7 kΩ pull-up resistor on the DS18B20 data line provides the required pull-up voltage for the open-drain OneWire bus. Three 220 Ω resistors limit the current through the indicator LEDs to safe operating levels.

1.6. Software Components

PlatformIO

PlatformIO is a professional open-source ecosystem for embedded development that integrates with Visual Studio Code as an extension. It provides comprehensive project management, automatic library dependency resolution, multi-board and multi-environment support, and unified build/upload/monitor tools. In this laboratory, PlatformIO manages the compilation environment

for the Arduino Mega 2560, automatically resolves library dependencies (FreeRTOS, OneWire, DallasTemperature, LiquidCrystal_I2C), and provides the serial monitor for observing STDIO output.

The PlatformIO project configuration (`platformio.ini`) defines a dedicated environment for Lab 3.1 with the required build flags and library dependencies, enabling the project to maintain multiple lab configurations simultaneously without code conflicts whether to run one lab environment or another.

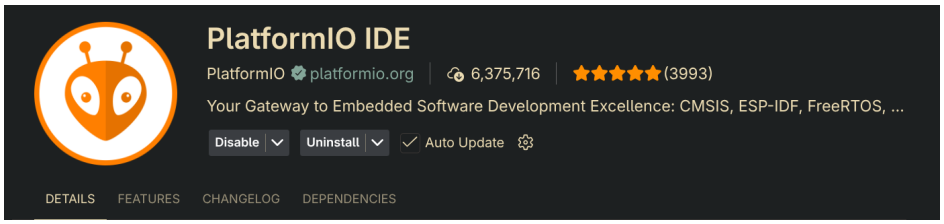

***Figure 1.7** – PlatformIO IDE in VS Code with Lab 3.1 project*

Wokwi Simulator

Wokwi is an embedded systems simulator available both as a web application and as a VS Code extension. It supports Arduino, ESP32, Raspberry Pi Pico, and other popular platforms, providing virtual components including LEDs, buttons, LCDs, temperature sensors (NTC thermistors and DS18B20), and serial terminals. Wokwi enables testing of the complete application in a simulated environment before deploying to physical hardware, which is particularly valuable for temperature sensor applications where controlled temperature variation would otherwise require physical heating/cooling equipment.

In this laboratory, Wokwi simulates the complete circuit: the Arduino Mega 2560, an NTC thermistor with adjustable temperature, a DS18B20 sensor with adjustable temperature, a 16x2 I2C LCD display, three LEDs with resistors, and the voltage divider and pull-up resistors. The simulator allows interactive temperature adjustment during runtime, making it possible to test threshold crossings, hysteresis behavior, and debounce filtering without physical sensor manipulation.

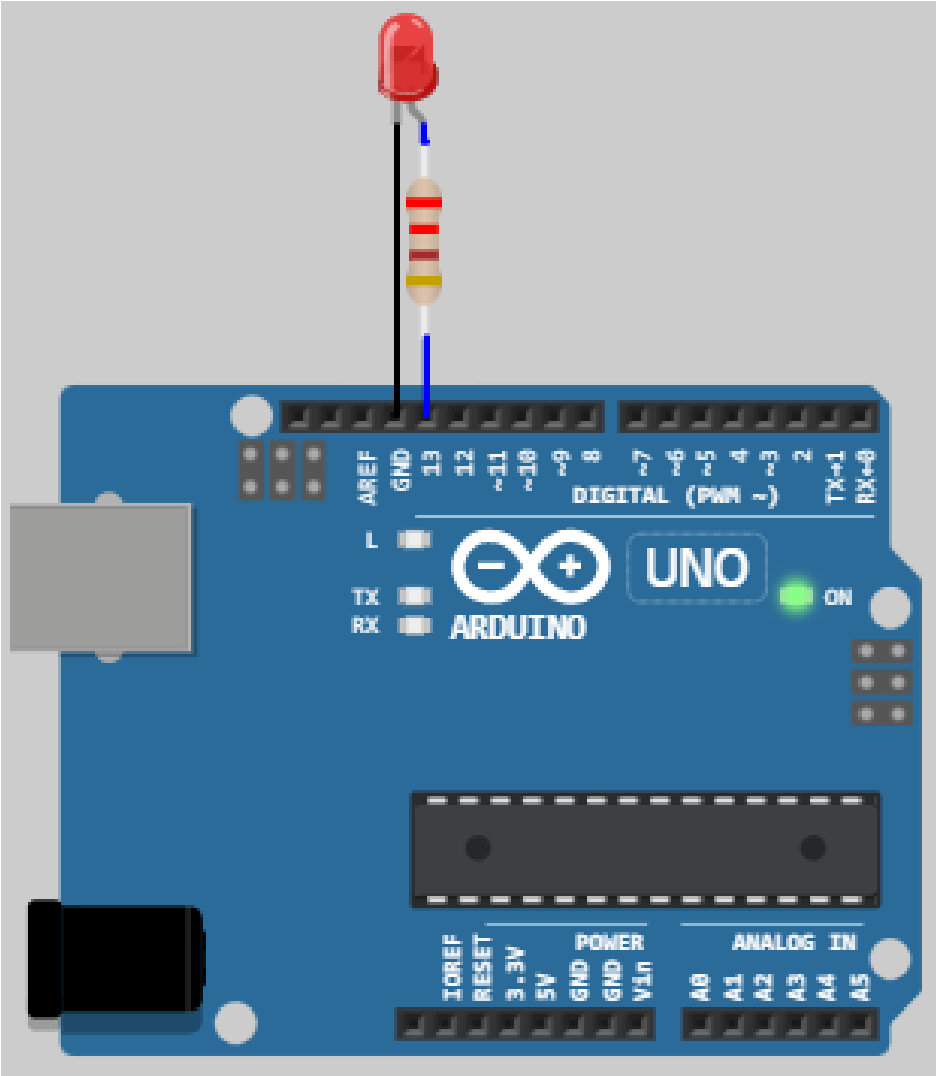

Figure 1.8 *– Wokwi simulator running the Lab 3.1 circuit*

1.7. Case Study: Industrial Temperature Monitoring Systems

The dual-sensor temperature monitoring architecture implemented in this laboratory closely mirrors industrial temperature monitoring systems used in manufacturing, food processing, HVAC, and data center environments. In these real-world applications, redundant sensor configurations (combining analog and digital sensors) provide both measurement diversity and fault tolerance.

In pharmaceutical cold chain monitoring, for example, regulatory requirements mandate continuous temperature logging with alarm capabilities. Pharmaceutical storage facilities use multiple temperature sensors per zone, with hysteresis-based alerting to prevent nuisance alarms during door openings while ensuring that sustained temperature excursions trigger immediate notifications. The signal conditioning pipeline implemented in this lab — acquisition, threshold evaluation with hysteresis, debounce filtering, and multi-channel reporting — directly corresponds to the processing stages found in commercial pharmaceutical monitoring systems, albeit at a smaller scale.

Data center thermal management represents another relevant application. Server rooms require precise environmental monitoring to prevent equipment overheating while avoiding unnec-

essary cooling that increases energy costs. Modern data center monitoring systems deploy hundreds of temperature sensors (both thermistors for air temperature and digital sensors integrated into server motherboards), process their readings through configurable threshold engines, and generate alerts through dashboards and notification systems. The FreeRTOS-based task architecture used in this lab demonstrates the same separation of concerns found in these larger systems: dedicated acquisition tasks for sensor polling, middleware conditioning tasks for threshold logic, and presentation tasks for human-machine interface updates.

The hysteresis technique implemented in the ThresholdAlert module is particularly important in automotive Engine Control Units (ECUs), where coolant temperature monitoring must balance sensitivity (detecting genuine overheating conditions rapidly) with stability (avoiding false alarms during normal thermal cycling). Automotive ECUs typically employ 2–5°C hysteresis bands with multi-sample debounce windows, closely matching the approach used in this laboratory.

2. Design

2.1. System Architecture Diagrams

Component-Level Architecture

The dual-sensor temperature monitoring system connects seven hardware components to the Arduino Mega 2560 microcontroller through four distinct interfaces: ADC (analog), OneWire (digital), I2C (LCD), and GPIO (LEDs). The component-level architecture diagram in *Figure 2.1* illustrates the physical topology and communication protocols between all system elements.

The NTC thermistor connects through a voltage divider to analog pin A0, providing a continuously variable voltage that the ADC samples at each acquisition cycle. The DS18B20 connects through a single OneWire data line (with a 4.7 kΩ pull-up resistor) to digital pin 2, communicating temperature readings as digital data packets. The LCD 1602 connects via the I2C bus (SDA on pin 20, SCL on pin 21), allowing the display to be updated with formatted temperature and status information. Three LEDs connect through current-limiting resistors to digital pins 8, 9, and 10, providing immediate visual feedback on system status and alert conditions.

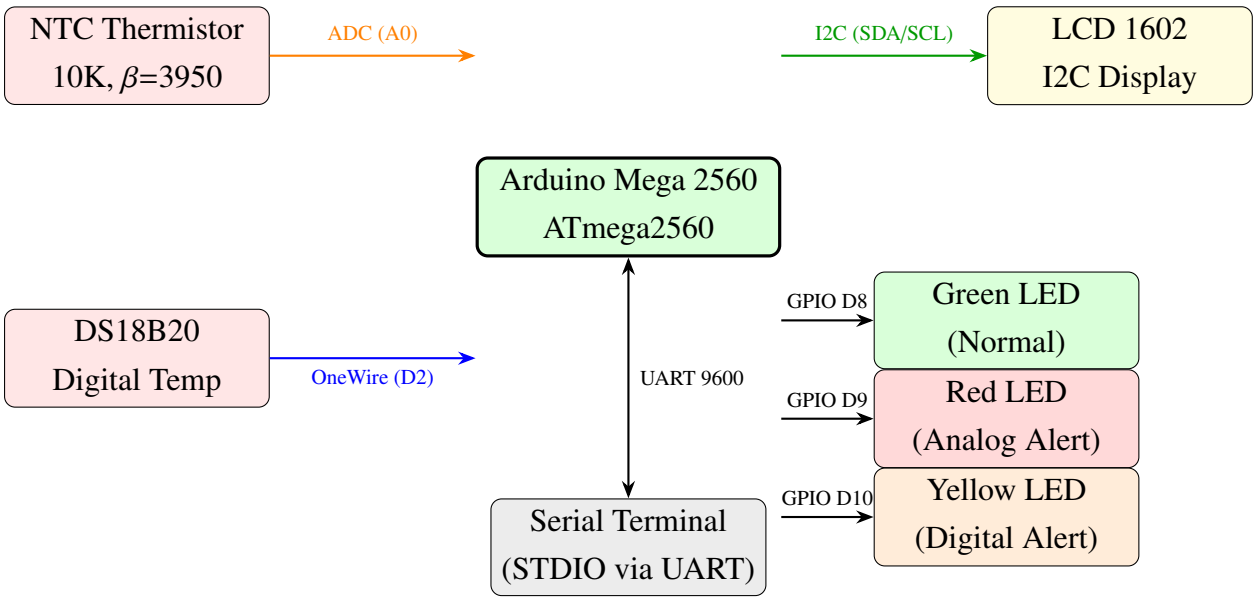

Figure 2.1 *– System structural diagram — component-level architecture*

The data flows in the system follow distinct paths. Sensor data flows inward from the NTC thermistor and DS18B20 into the microcontroller, where it is processed by the acquisition and conditioning pipeline. Processed results flow outward through two channels: the I2C bus to the LCD for real-time display, and the UART serial port for detailed STDIO reports. Control signals flow outward through GPIO to the LEDs for immediate visual status indication.

Layered System Architecture

The software architecture follows a layered design pattern that separates hardware-specific drivers from application logic, enabling code reuse across laboratory projects and simplifying maintenance. *Figure 2.2* illustrates the five-layer hierarchy used in this system.

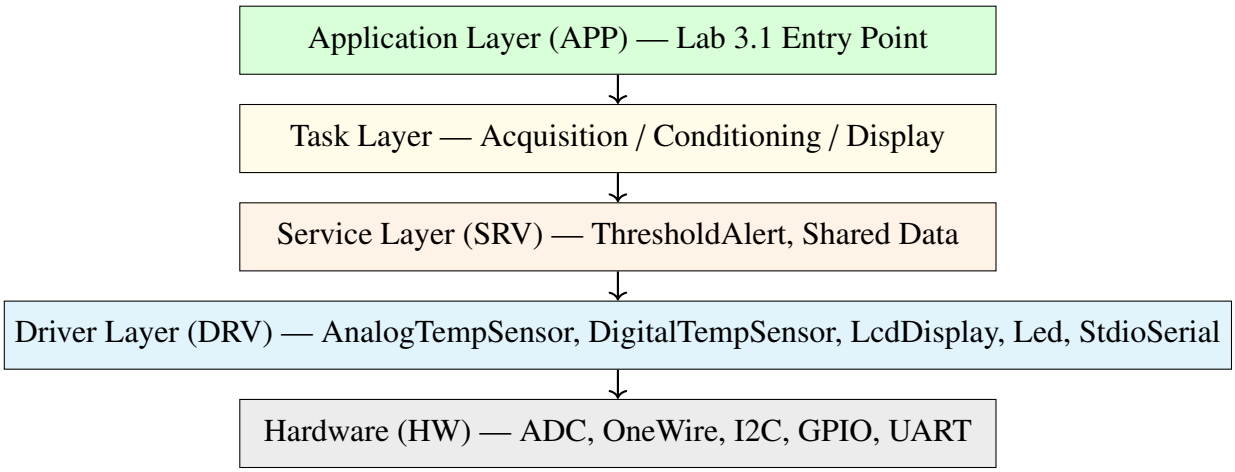

Figure 2.2 *– Layered software architecture of the dual-sensor monitoring system*

The Hardware Layer (HW) encompasses the physical peripherals: the ADC for the NTC thermistor, the OneWire bus for the DS18B20, the I2C bus for the LCD, GPIO pins for the LEDs,

and the UART for serial communication. The Driver Layer (DRV) contains reusable library modules that abstract hardware access behind clean C++ interfaces. The Service Layer (SRV) provides application-independent processing logic, including the ThresholdAlert module for hysteresis detection and the shared data structures with their synchronization primitives. The Task Layer defines the three FreeRTOS tasks that implement the monitoring pipeline. The Application Layer (APP) is the lab entry point that initializes hardware, creates synchronization primitives, spawns tasks, and starts the FreeRTOS scheduler.

Each layer communicates only with its immediate neighbor, ensuring that application code never directly manipulates hardware registers, and driver modules can be tested and reused independently of the specific application logic.

FreeRTOS Task Architecture

The three FreeRTOS tasks operate as a pipeline, with well-defined data flows and synchronization points. *Figure 2.3* shows the task communication topology.

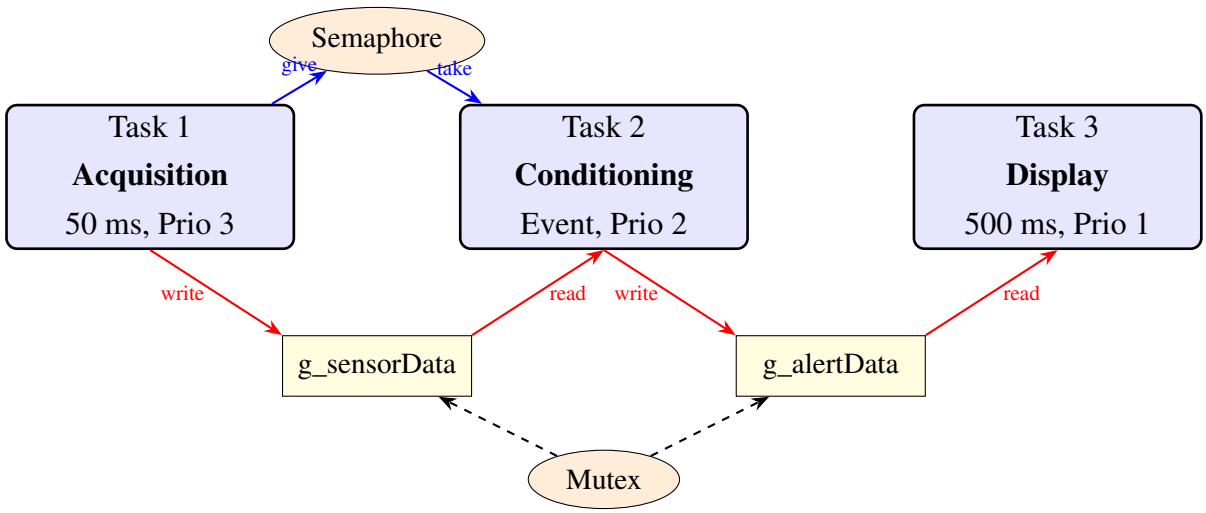

***Figure 2.3** – FreeRTOS task communication architecture*

Task 1 (Acquisition) runs at the highest priority (3) with a 50 ms period using `vTaskDelayUntil()` for drift-free timing. It reads both sensors, writes the results to `g_sensorData` under mutex protection, and gives the binary semaphore to notify Task 2. Task 2 (Conditioning) runs at medium priority (2) and blocks on the semaphore until new data is available, then reads the sensor data, applies the threshold detection logic for both sensors, writes the alert status to `g_alertData` under mutex protection, and updates the LED indicators. Task 3 (Display) runs at the lowest priority (1) with a 500 ms period, reading both shared data structures under mutex protection and formatting the output for the LCD and STDIO.

2.2. Block Diagrams

Main Application Flowchart

Figure 2.4 shows the overall initialization sequence performed by the `lab3_1Setup()` function. After initialization completes, the FreeRTOS scheduler takes over and the three tasks run concurrently.

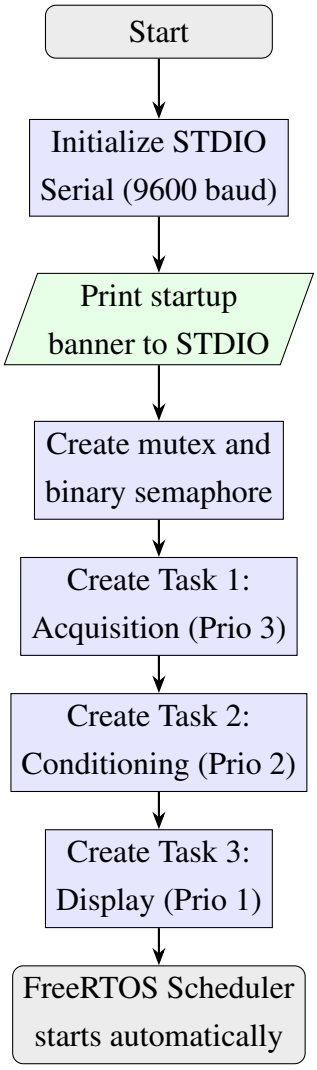

Figure 2.4 *– Main application initialization flowchart (`lab3_1Setup`)*

Sensor Acquisition Task Flowchart

The acquisition task (*Figure 2.5*) implements the periodic reading of both temperature sensors. It uses a non-blocking approach for the DS18B20: on each cycle, it reads the result of the previous conversion and starts a new one, overlapping the conversion time with the next sleep period.

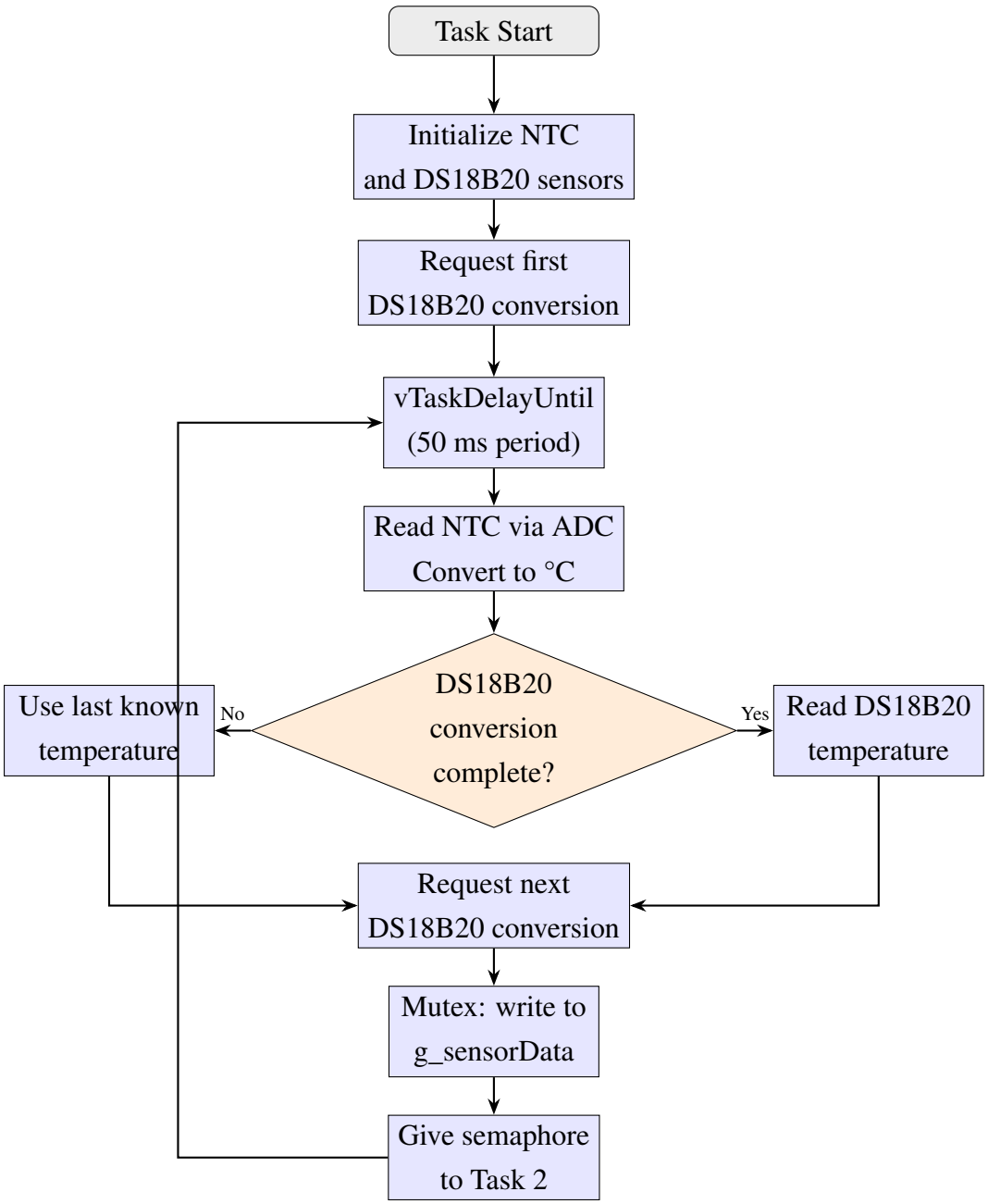

Figure 2.5 *– Sensor acquisition task flowchart (Task 1)*

The NTC reading is fast (a single `analogRead()` call takes approximately 104 μs), while the DS18B20 conversion runs asynchronously. By starting the conversion at the end of each cycle and reading the result at the beginning of the next, the conversion time is hidden behind the 50 ms sleep period, maintaining consistent timing.

Signal Conditioning Task Flowchart

The conditioning task (*Figure 2.6*) is event-driven, waking only when the acquisition task signals new data via the binary semaphore. This approach eliminates polling overhead and ensures that conditioning always processes the most recent sensor data.

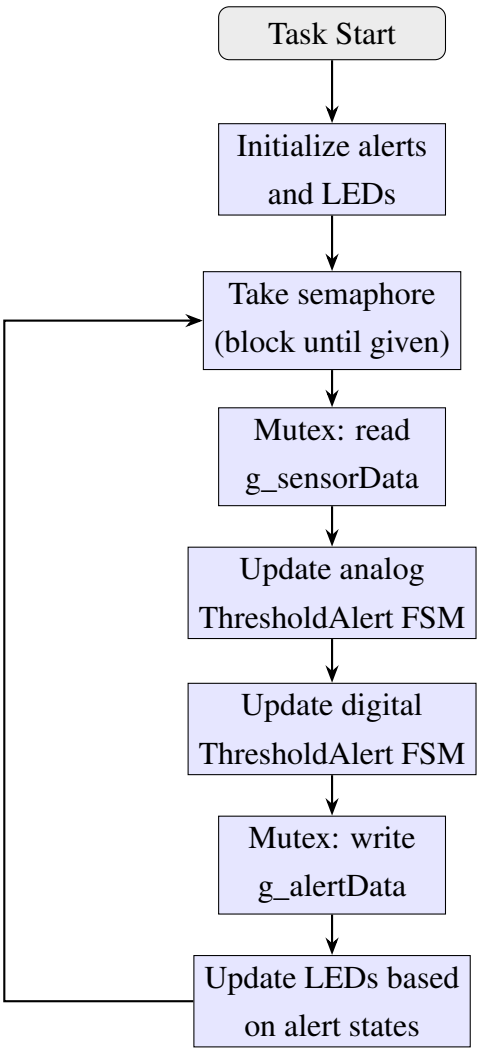

Figure 2.6 *– Signal conditioning and alerting task flowchart (Task 2)*

Threshold Alert FSM State Diagram

The ThresholdAlert module implements a 4-state finite state machine (FSM) that governs the transition between normal and alert conditions. *Figure 2.7* shows the state diagram with transition conditions.

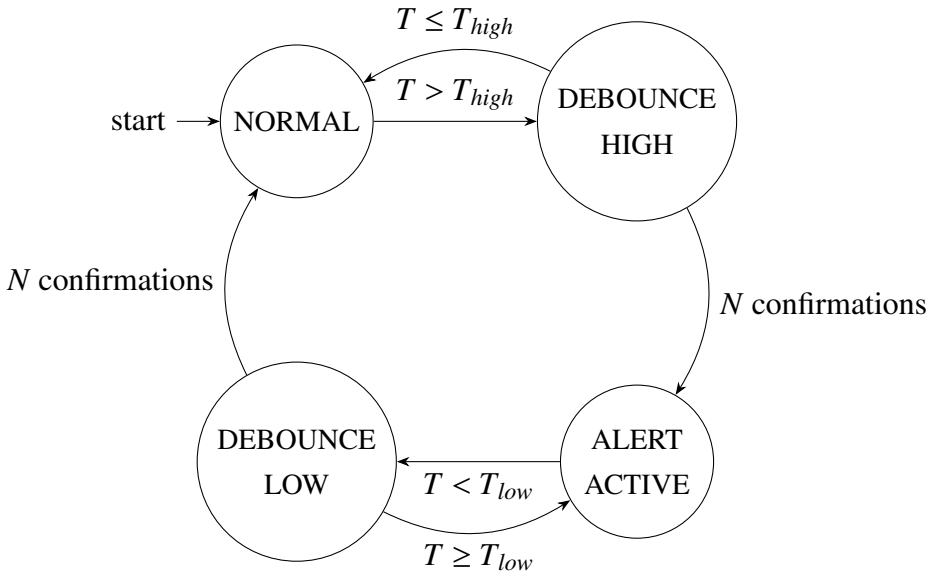

***Figure 2.7** – Threshold alert FSM state diagram with hysteresis and debounce*

In the NORMAL state, the system monitors the input value and transitions to DEBOUNCE_HIGH when the value exceeds the high threshold (T_{high} = 30°C). The DEBOUNCE_HIGH state counts consecutive readings above the threshold; after N confirmations (default 5), the system commits to the ALERT_ACTIVE state. If the value drops back below the threshold before N confirmations, the transition is cancelled and the system returns to NORMAL. Symmetrically, from ALERT_ACTIVE, the system transitions to DEBOUNCE_LOW when the value drops below the low threshold (T_{low} = 28°C), and returns to NORMAL after N confirmed readings below the threshold. This dual-layer protection ensures that both transient noise (filtered by debounce) and sustained oscillation near a single threshold (filtered by hysteresis) are handled robustly.

Display Task Flowchart

The display task (*Figure 2.8*) runs periodically at 500 ms intervals, reading the latest sensor data and alert status to update both the LCD display and the serial terminal output. The LCD alternates between two information pages, and the STDIO report is generated every 4th cycle (2 seconds).

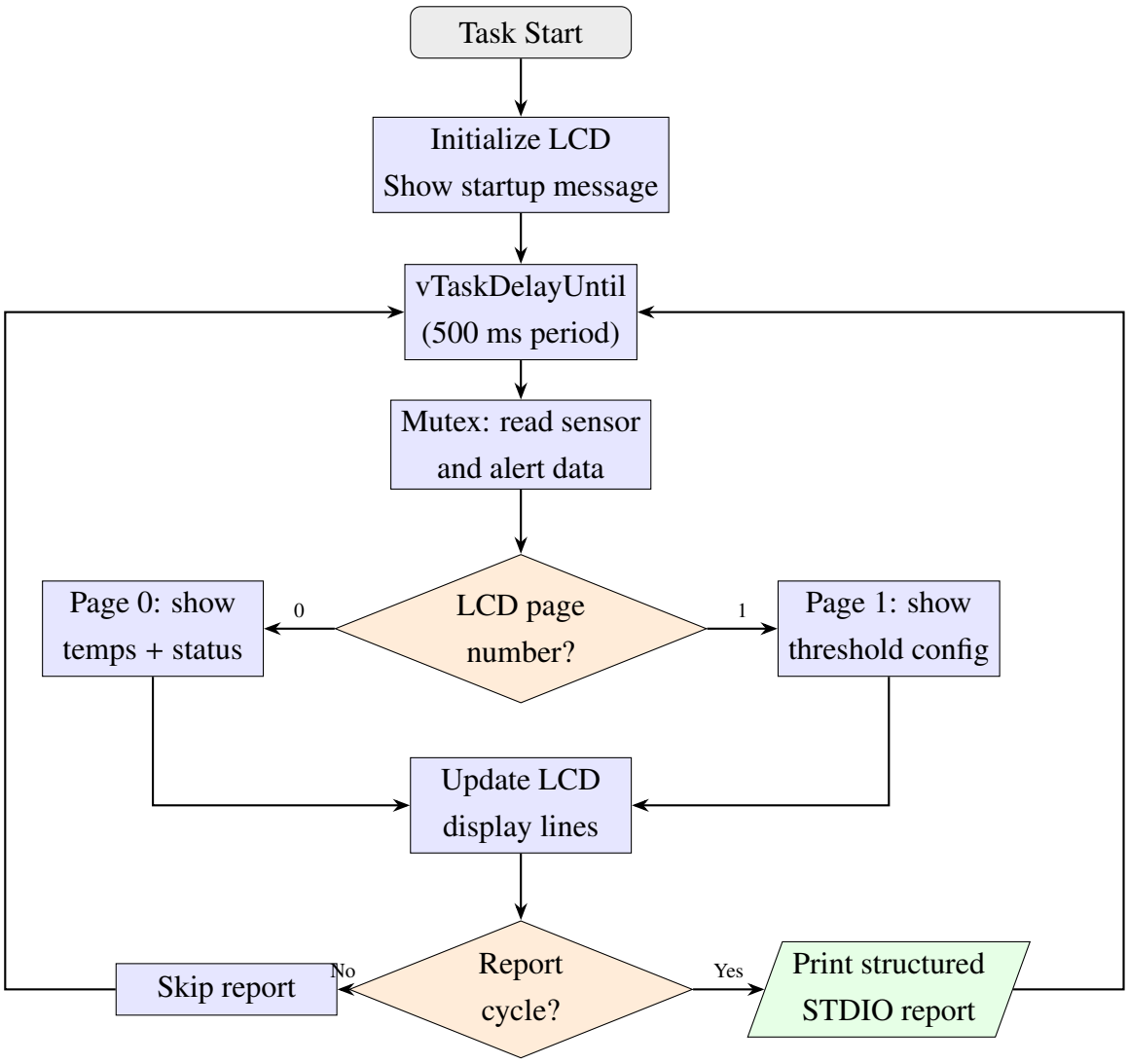

Figure 2.8 *– Display and reporting task flowchart (Task 3)*

2.3. Electrical Schematics

The complete circuit schematic (*Figure 2.9*) shows all electrical connections between the Arduino Mega 2560 and peripheral components.

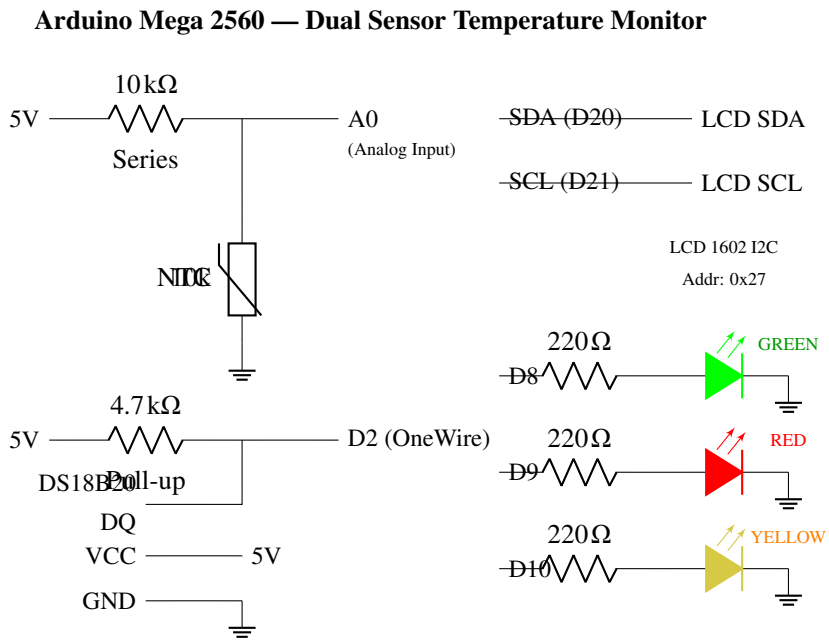

Figure 2.9 *– Complete circuit schematic of the dual-sensor monitoring system*

Component Specification

- **NTC Thermistor**: 10 kΩ nominal at 25°C, Beta coefficient β = 3950 K, operating range −40°C to +125°C.
- **DS18B20**: Digital temperature sensor, −55°C to +125°C range, ±0.5°C accuracy (−10 to +85°C), 10-bit resolution (0.25°C), OneWire interface.
- **Series Resistor (NTC)**: 10 kΩ, 1/4 W, ±5% tolerance. Forms voltage divider with the NTC thermistor.
- **Pull-up Resistor (DS18B20)**: 4.7 kΩ, 1/4 W. Required for OneWire open-drain bus operation.
- **LED Current-Limiting Resistors**: 3 × 220 Ω, 1/4 W. Limits LED forward current to approximately 15 mA.
- **LEDs**: Standard 5 mm, green (570 nm, 2.1 V V_f), red (620 nm, 2.0 V V_f), yellow (590 nm, 2.1 V V_f).
- **LCD 1602**: 16×2 character display, HD44780-compatible, PCF8574 I2C backpack (address 0x27), 5V operation.

Circuit Connections

The complete pin mapping for the system is summarized in the following table:

Arduino Pin	Component	Interface	Description
A0	NTC Thermistor (via divider)	ADC	Analog temperature reading
D2	DS18B20 DQ	OneWire	Digital temperature data
D8	Green LED (220Ω)	GPIO	Normal status indicator
D9	Red LED (220Ω)	GPIO	Analog alert indicator
D10	Yellow LED (220Ω)	GPIO	Digital alert indicator
D20	LCD SDA	I2C	Display data line
D21	LCD SCL	I2C	Display clock line
5V	NTC divider, DS18B20, LCD	Power	Component power supply
GND	All components	Power	Common ground reference

Table 1: Complete pin mapping for Lab 3.1

Hardware Configuration

The Wokwi simulation configuration file defines the virtual firmware path and debug symbols:

Listing 2.1 *– Wokwi configuration file (wokwi.toml)*

```
[wokwi]
version = 1
firmware = "../../.pio/build/lab3_1/firmware.hex"
elf = "../../.pio/build/lab3_1/firmware.elf"
```

[Screenshot pending — run the Wokwi simulation and save to resources/Design/ElectricalSchematics/wokwi_circuit.png]

Figure 2.10 *– Wokwi simulation circuit (active run)*

2.4. Project Structure

The project follows the established repository convention, with lab-specific code in a dedicated folder and reusable libraries shared across labs:

Listing 2.2 *– Project directory structure for Lab 3.1*

```
labs/
|-- platformio.ini
|-- src/
|   |-- main.cpp                    (Lab selector entry point)
|-- lab/
|   |-- lab3_1/
|       |-- lab3_1_main.cpp         (Lab entry point)
|       |-- lab3_1_main.h           (Lab entry point header)
|       |-- sensor_data.h           (Shared data + sync primitives)
|       |-- sensor_data.cpp         (Shared data definitions)
```

```
|       |-- task_acquisition.h      (Task 1 interface)
|       |-- task_acquisition.cpp    (Task 1 implementation)
|       |-- task_conditioning.h     (Task 2 interface)
|       |-- task_conditioning.cpp   (Task 2 implementation)
|       |-- task_display.h          (Task 3 interface)
|       |-- task_display.cpp        (Task 3 implementation)
|-- lib/
|   |-- AnalogTempSensor/           (NTC thermistor driver)
|   |   |-- AnalogTempSensor.h
|   |   |-- AnalogTempSensor.cpp
|   |-- DigitalTempSensor/          (DS18B20 OneWire driver)
|   |   |-- DigitalTempSensor.h
|   |   |-- DigitalTempSensor.cpp
|   |-- ThresholdAlert/             (Hysteresis + debounce module)
|   |   |-- ThresholdAlert.h
|   |   |-- ThresholdAlert.cpp
|   |-- LcdDisplay/                 (I2C LCD driver, reused)
|   |   |-- LcdDisplay.h
|   |   |-- LcdDisplay.cpp
|   |-- Led/                        (LED driver, reused)
|   |   |-- Led.h
|   |   |-- Led.cpp
|   |-- StdioSerial/                (STDIO redirection, reused)
|       |-- StdioSerial.h
|       |-- StdioSerial.cpp
|-- wokwi/
    |-- lab3.1/
        |-- diagram.json            (Wokwi circuit definition)
        |-- wokwi.toml              (Wokwi firmware path)
```

This structure reflects the layered architecture: reusable driver-layer modules reside in `lib/`, while lab-specific task and application code resides in `lab/lab3_1/`. The separation ensures that sensor drivers and the threshold alert module can be reused in future labs without modification.

2.5. Modular Implementation

Driver Layer: AnalogTempSensor (NTC Thermistor)

The AnalogTempSensor module provides a clean interface for reading temperature from an NTC thermistor connected through a voltage divider. The module encapsulates all ADC interaction and the Steinhart-Hart Beta parameter equation, presenting callers with simple methods like `readTemperatureC()`.

Listing 2.3 *– AnalogTempSensor interface (AnalogTempSensor.h)*

```
class AnalogTempSensor {
public:
```

```
    AnalogTempSensor(uint8_t adcPin, uint32_t seriesR,
                     uint32_t nominalR, uint16_t betaCoeff,
                     float nominalTempC = 25.0f,
                     uint16_t adcResolution = 10);
    void init();
    uint16_t readRaw();
    float readResistance();
    float readTemperatureC();
    float readTemperatureF();
    bool isValid() const;
    uint16_t getLastRaw() const;
    float getLastTemperatureC() const;
};
```

The constructor accepts all configuration parameters specific to the thermistor and voltage divider circuit: the ADC pin number, series resistance, nominal NTC resistance and temperature, Beta coefficient, and ADC resolution. This design allows the same module to work with different thermistors and circuit configurations simply by changing the constructor arguments. The `readResistance()` method calculates the thermistor resistance from the ADC reading using the voltage divider equation, including validation guards against division by zero (ADC reads 0 or maximum, indicating a short or open circuit). The `readTemperatureC()` method chains the ADC reading, resistance calculation, and Beta equation conversion in a single call.

Listing 2.4 *– Key implementation — NTC temperature conversion*

```
float AnalogTempSensor::readTemperatureC() {
    float resistance = readResistance();
    if (resistance < 0.0f) {
        _lastTempC = NAN;
        _valid = false;
        return NAN;
    }
    // Steinhart-Hart Beta equation:
    // 1/T = 1/T0 + (1/B) * ln(R / R0)
    float steinhart = log(resistance / (float)_nominalR);
    steinhart /= (float)_betaCoeff;
    steinhart += 1.0f / _nominalTempK;
    float tempK = 1.0f / steinhart;
    _lastTempC = tempK - 273.15f;
    _valid = true;
    return _lastTempC;
}
```

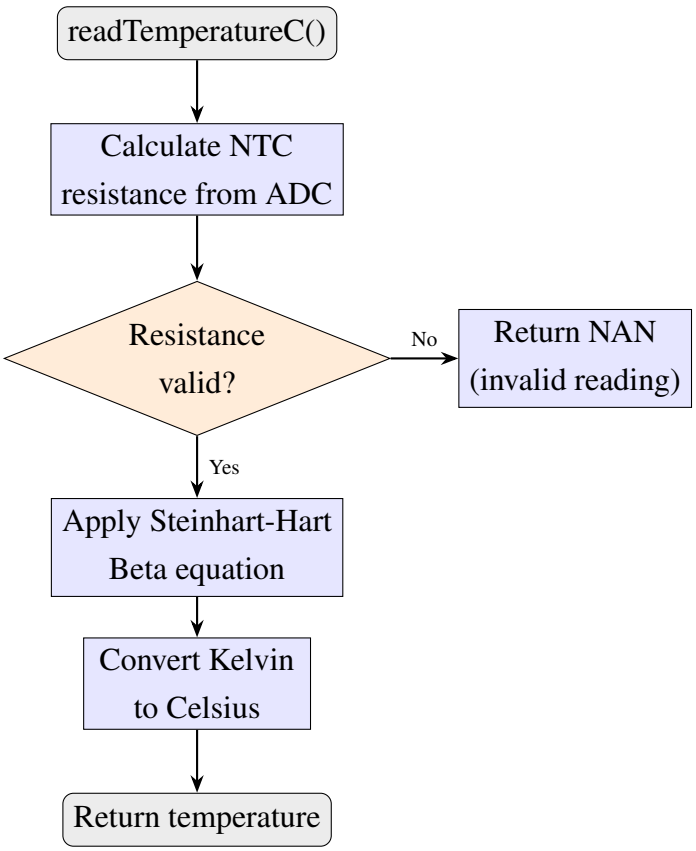

Figure 2.11 *– Flowchart: AnalogTempSensor::readTemperatureC()*

Driver Layer: DigitalTempSensor (DS18B20)

The DigitalTempSensor module abstracts the OneWire protocol and DallasTemperature library behind an interface consistent with the AnalogTempSensor API. This consistency simplifies the acquisition task code, which can call `readTemperatureC()` on both sensor objects regardless of their underlying communication protocols.

Listing 2.5 *– DigitalTempSensor interface (DigitalTempSensor.h)*

```
class DigitalTempSensor {
public:
    DigitalTempSensor(uint8_t dataPin,
                      uint8_t resolution = 10);
    bool init();
    void requestConversion();
    bool isConversionComplete();
    float readTemperatureC();
    float getLastTemperatureC() const;
    float readTemperatureF();
    bool isConnected() const;
    bool isValid() const;
    uint8_t getDeviceCount() const;
};
```

The `init()` method scans the OneWire bus for DS18B20 devices, sets the measurement

resolution, and returns a boolean indicating whether at least one sensor was found. The `requestConversion()` and `isConversionComplete()` methods support non-blocking operation: the acquisition task can start a conversion, continue other work, and check for completion later. The `readTemperatureC()` method provides a simpler blocking alternative that handles the entire conversion cycle internally.

***Listing 2.6** – Key implementation — DS18B20 initialization and reading*

```
bool DigitalTempSensor::init() {
    _sensors.begin();
    _deviceCount = _sensors.getDeviceCount();
    if (_deviceCount == 0) {
        _connected = false;
        return false;
    }
    _connected = true;
    _sensors.setResolution(_resolution);
    _sensors.setWaitForConversion(false);
    return true;
}

float DigitalTempSensor::readTemperatureC() {
    if (!_connected) {
        _valid = false;
        _lastTempC = NAN;
        return NAN;
    }
    _sensors.setWaitForConversion(true);
    _sensors.requestTemperatures();
    _sensors.setWaitForConversion(false);
    float tempC = _sensors.getTempCByIndex(0);
    if (tempC == DEVICE_DISCONNECTED_C ||
        tempC <= DISCONNECTED_TEMP) {
        _valid = false;
        _lastTempC = NAN;
        return NAN;
    }
    if (tempC == POWER_ON_RESET_TEMP) {
        _valid = false;
        return _lastTempC;
    }
    _valid = true;
    _lastTempC = tempC;
    return _lastTempC;
}
```

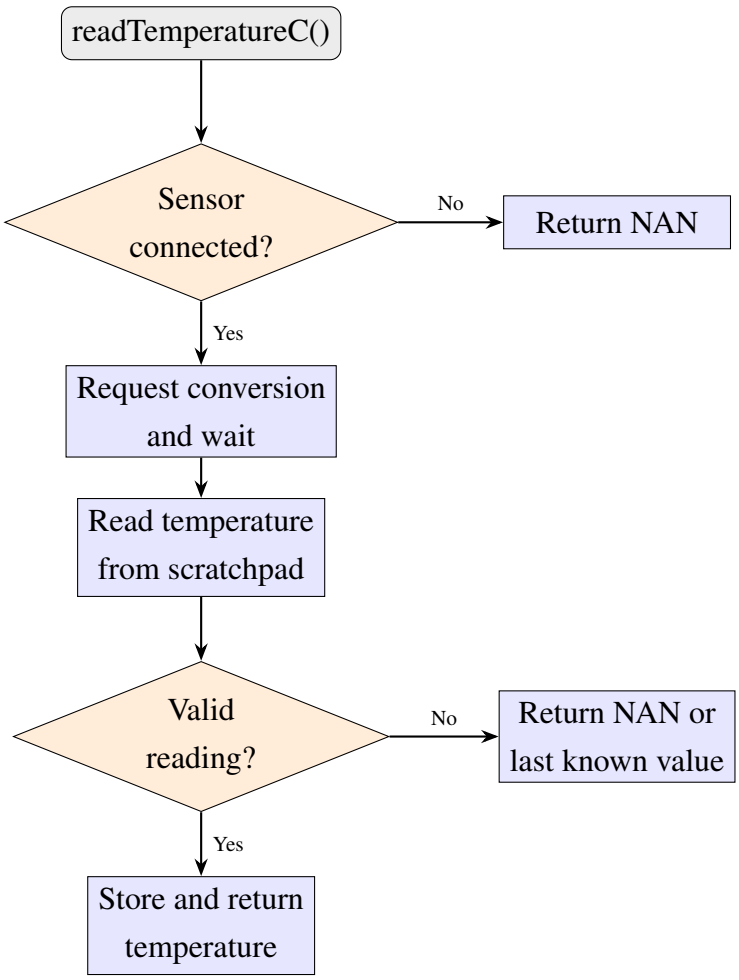

Figure 2.12 *– Flowchart: DigitalTempSensor::readTemperatureC()*

Service Layer: ThresholdAlert Module

The ThresholdAlert module is the central signal conditioning component. It implements the 4-state FSM (NORMAL → DEBOUNCE_HIGH → ALERT_ACTIVE → DEBOUNCE_LOW → NORMAL) with configurable thresholds and debounce count. Two independent instances of this module are created — one for the analog sensor and one for the digital sensor — demonstrating the reusable design.

Listing 2.7 *– ThresholdAlert interface (ThresholdAlert.h)*

```
enum AlertState {
    ALERT_NORMAL,
    ALERT_DEBOUNCE_HIGH,
    ALERT_ACTIVE,
    ALERT_DEBOUNCE_LOW
};

class ThresholdAlert {
public:
    ThresholdAlert(float highThreshold,
                   float lowThreshold,
```

```
                    uint8_t debounceCount = 5);
    void init();
    AlertState update(float value);
    AlertState getState() const;
    bool isAlertActive() const;
    bool isDebouncing() const;
    uint8_t getDebounceCounter() const;
    const char* getStateString() const;
    void setThresholds(float high, float low);
    void setDebounceCount(uint8_t count);
    float getHighThreshold() const;
    float getLowThreshold() const;
};
```

The `update()` method is the core of the module. It accepts a new sensor reading and advances the FSM based on the current state and the relationship between the input value and the configured thresholds.

Listing 2.8 *– ThresholdAlert::update() — core FSM logic*

```
AlertState ThresholdAlert::update(float value) {
    switch (_state) {
    case ALERT_NORMAL:
        if (value > _highThreshold) {
            _state = ALERT_DEBOUNCE_HIGH;
            _debounceCounter = 1;
        }
        break;
    case ALERT_DEBOUNCE_HIGH:
        if (value > _highThreshold) {
            _debounceCounter++;
            if (_debounceCounter >= _debounceMax) {
                _state = ALERT_ACTIVE;
                _debounceCounter = 0;
            }
        } else {
            _state = ALERT_NORMAL;
            _debounceCounter = 0;
        }
        break;
    case ALERT_ACTIVE:
        if (value < _lowThreshold) {
            _state = ALERT_DEBOUNCE_LOW;
            _debounceCounter = 1;
        }
        break;
    case ALERT_DEBOUNCE_LOW:
        if (value < _lowThreshold) {
            _debounceCounter++;
```

```
            if (_debounceCounter >= _debounceMax) {
                _state = ALERT_NORMAL;
                _debounceCounter = 0;
            }
        } else {
            _state = ALERT_ACTIVE;
            _debounceCounter = 0;
        }
        break;
    }
    return _state;
}
```

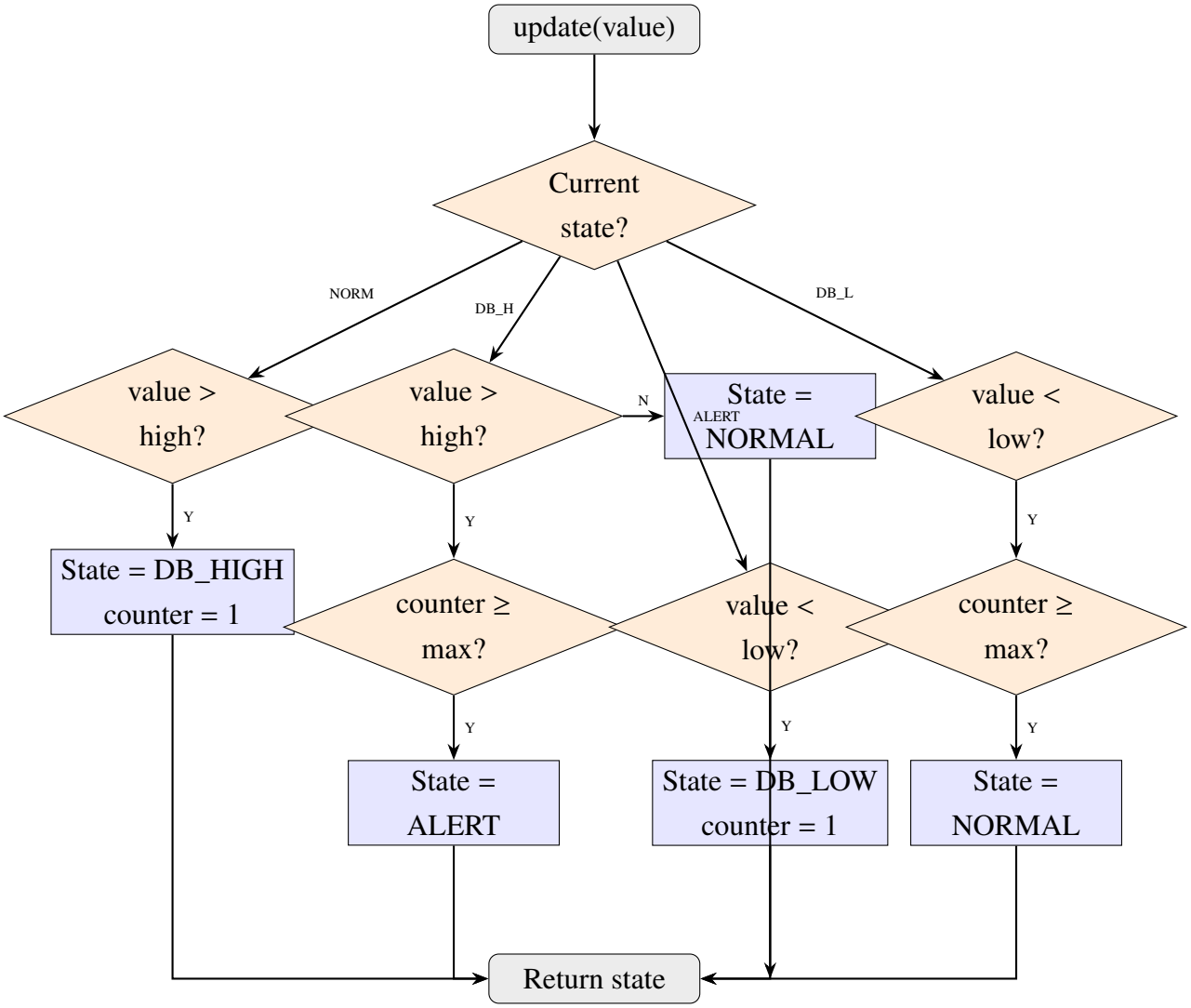

Figure 2.13 *– Flowchart: ThresholdAlert::update() FSM transitions*

Driver Layer: LcdDisplay (Reused)

The LcdDisplay module was originally developed for Lab 1.2 and is reused without modification. It wraps the LiquidCrystal_I2C library behind a simplified interface focused on line-based text operations, which is sufficient for the status display requirements of this lab.

Listing 2.9 – LcdDisplay interface (LcdDisplay.h) — reused from Lab 1.2

```
class LcdDisplay {
public:
    LcdDisplay(uint8_t i2cAddress, uint8_t cols,
               uint8_t rows);
    void init();
    void clear();
    void setCursor(uint8_t col, uint8_t row);
    void print(const char *text);
    void printLine(uint8_t row, const char *text);
    void showTwoLines(const char *line1,
                      const char *line2);
    void backlight(bool on);
};
```

The `showTwoLines()` method is the primary API used by the display task, clearing both lines and writing new content in a single call. The `printLine()` method pads shorter strings with spaces to ensure that previous content is fully overwritten without requiring a full `clear()` command (which causes visible flicker on the LCD).

Driver Layer: Led (Reused)

The Led module was originally developed for Lab 1.1 and is reused in this lab to control the three indicator LEDs. The module provides a minimal API for initialization, turning on/off, toggling, and querying the current state.

Listing 2.10 – Led interface (Led.h) — reused from Lab 1.1

```
class Led {
public:
    Led(uint8_t pin);
    void init();
    void turnOn();
    void turnOff();
    void toggle();
    bool isOn() const;
};
```

In the conditioning task, the green LED uses `turnOn()` and `turnOff()` to indicate normal/alert status, while the red and yellow LEDs use `toggle()` during debounce transitions to produce a blinking effect that visually distinguishes the transient debounce states from the stable NORMAL and ALERT states.

Driver Layer: StdioSerial (Reused)

The StdioSerial module redirects the C standard I/O streams (`stdout`, `stdin`) to the hardware UART serial port. This enables the use of `printf()` for formatted output in the display task,

producing the structured diagnostic reports that include raw ADC values, resistance calculations, temperatures, alert states, and cumulative statistics.

Listing 2.11 *– StdioSerial interface (StdioSerial.h) — reused from Lab 1.1*

```
void stdioSerialInit(unsigned long baudRate);
```

The module uses AVR libc's `fdev_setup_stream()` to create a custom FILE stream backed by the Arduino Serial (UART) hardware. After calling `stdioSerialInit(9600)`, all subsequent `printf()` calls are transparently routed to the serial port at 9600 baud.

Service Layer: Shared Data and Synchronization

The sensor_data module defines the shared data structures and FreeRTOS synchronization primitives that enable safe inter-task communication. Two structures are defined: `SensorReadings_t` holds the raw and converted sensor values written by Task 1, and `AlertStatus_t` holds the threshold alert states and statistics written by Task 2.

Listing 2.12 *– Shared data structures (sensor_data.h — excerpt)*

```
typedef struct {
    uint16_t analogRaw;
    float    analogResistance;
    float    analogTempC;
    bool     analogValid;
    float    digitalTempC;
    bool     digitalValid;
    uint32_t readingCount;
    TickType_t timestamp;
} SensorReadings_t;

typedef struct {
    AlertState analogAlertState;
    uint8_t    analogDebounceCount;
    float      analogFilteredTemp;
    AlertState digitalAlertState;
    uint8_t    digitalDebounceCount;
    float      digitalFilteredTemp;
    uint32_t   analogAlertCount;
    uint32_t   digitalAlertCount;
    uint32_t   conditioningCycles;
} AlertStatus_t;
```

A binary semaphore signals Task 2 when new sensor data is available, and a mutex with priority inheritance protects both shared structures from concurrent access. The `sensorDataInit()` function creates these primitives and must be called before any tasks are created.

Application Layer: Lab 3.1 Entry Point

The lab entry point (`lab3_1_main.cpp`) orchestrates the entire system initialization sequence. It follows a strict order: STDIO initialization (for the startup banner), synchronization primitive creation (must precede task creation), and task spawning (with decreasing priority). The FreeRTOS scheduler starts automatically after `setup()` returns, at which point the three tasks begin executing according to their priorities and timing constraints.

Listing 2.13 *– Lab 3.1 entry point (lab3_1_main.cpp — excerpt)*

```
void lab3_1Setup() {
    stdioSerialInit(9600);
    printf("Lab 3.1 - Dual-Sensor Temp Monitor\r\n");
    // Print full startup banner...

    sensorDataInit();  // Create mutex + semaphore

    xTaskCreate(vTaskAcquisition, "Acquire",
                TASK_ACQUISITION_STACK, NULL,
                TASK_ACQUISITION_PRIORITY, NULL);
    xTaskCreate(vTaskConditioning, "Cond",
                TASK_CONDITIONING_STACK, NULL,
                TASK_CONDITIONING_PRIORITY, NULL);
    xTaskCreate(vTaskDisplay, "Display",
                TASK_DISPLAY_STACK, NULL,
                TASK_DISPLAY_PRIORITY, NULL);
}

void lab3_1Loop() {
    // All logic runs in FreeRTOS tasks.
}
```

The PlatformIO environment configuration for Lab 3.1 specifies the build flags and library dependencies:

Listing 2.14 *– PlatformIO environment for Lab 3.1 (platformio.ini — excerpt)*

```
[env:lab3_1]
platform = atmelavr
board = megaatmega2560
framework = arduino
monitor_speed = 9600
build_src_filter = +<*> +<../lab/lab3_1/*>
build_flags = -I lab/lab3_1 -DLAB3_1
lib_deps =
    feilipu/FreeRTOS
    paulstoffregen/OneWire@^2.3.8
    milesburton/DallasTemperature@^3.11.0
    marcoschwartz/LiquidCrystal_I2C@^1.1.4
```

3. Obtained Results

This section presents the compilation results, simulation behavior, and test scenarios executed on the dual-sensor temperature monitoring system. The results demonstrate that the implemented system correctly acquires temperature data from both analog and digital sensors, applies signal conditioning with hysteresis and debounce filtering, and presents the processed information through both the LCD display and structured STDIO reports.

3.1. Build Process

The project was compiled using PlatformIO for the Arduino Mega 2560 target platform. The build process includes compilation of all lab-specific source files, the three reusable libraries (AnalogTempSensor, DigitalTempSensor, ThresholdAlert), and three libraries reused from previous labs (LcdDisplay, Led, StdioSerial), along with the external dependencies (FreeRTOS, OneWire, DallasTemperature, LiquidCrystal_I2C).

[Screenshot pending — run `pio run -e lab3_1` and capture the terminal output showing `[SUCCESS]` Save to resources/ObtainedResults/build_results.png]

***Figure 3.1** – PlatformIO build output for Lab 3.1 environment*

The build completed successfully with the following resource utilization:

- **Flash memory**: 26 122 bytes (10.3% of 253 952 bytes available). The firmware size is dominated by the FreeRTOS kernel, the DallasTemperature and OneWire libraries, and the LiquidCrystal_I2C library.
- **RAM**: 1 969 bytes (24.0% of 8 192 bytes available). This includes the static globals, shared data structures, and the three task stacks (each pre-allocated by FreeRTOS). The ATmega2560's 8 KB of SRAM provides comfortable headroom for additional functionality.

The relatively modest resource consumption demonstrates that the modular architecture does not introduce significant overhead. The layered driver abstractions compile to efficient code because most methods are short and inline-friendly, and the C++ object model adds minimal runtime cost on AVR targets.

3.2. Simulation Startup

Upon starting the Wokwi simulation, the system executes the initialization sequence defined in `lab3_1Setup()`: STDIO serial initialization, synchronization primitive creation, and task spawning. The serial terminal displays the startup banner confirming that all components initialized correctly.

[Screenshot pending — start Wokwi simulation and capture the serial terminal showing the startup banner and initial readings. Save to resources/ObtainedResults/simulation_start.png]

***Figure 3.2** – Simulation startup — serial terminal showing initialization banner*

After startup, the green LED turns on immediately (indicating normal operation), while the red and yellow LEDs remain off (indicating no alert conditions). The LCD displays the initial temperature readings on Page 0, showing the analog and digital sensor values along with the "OK" status indicators.

3.3. Test Scenarios

Test 1: Normal Operation

In normal operation, both sensors report temperatures below the 30°C high threshold. The system displays stable readings, the green LED remains solidly on, and both alert indicators (red and yellow LEDs) remain off.

[Screenshot pending — show Wokwi simulation with both sensors at room temperature (25°C). Green LED on, Red/Yellow LEDs off. LCD showing "A:25.0C OK D:24.0C OK". Save to resources/ObtainedResults/test_normal_operation.png]

***Figure 3.3** – Normal operation — both sensors below threshold*

The STDIO report during normal operation shows all fields populated with valid data, alert states showing "NORMAL" for both sensors, and debounce counters at zero.

Test 2: Analog Sensor Alert Trigger

When the NTC thermistor temperature rises above 30°C (simulated by adjusting the NTC slider in Wokwi), the system transitions through the following sequence:

1. The ThresholdAlert FSM for the analog channel moves from NORMAL to DEBOUNCE_HIGH.
2. The red LED begins blinking (toggling each conditioning cycle) during the debounce phase.
3. After 5 consecutive readings above 30°C, the FSM transitions to ALERT_ACTIVE.
4. The red LED turns on solidly, and the green LED turns off.
5. The LCD status changes from "OK" to "!!" for the analog channel.

[Screenshot pending — set NTC slider above 30°C in Wokwi.
Red LED on, Green LED off, Yellow LED off.
LCD showing "A:32.5C !! D:24.0C OK".
Save to resources/ObtainedResults/test_analog_alert.png]

***Figure 3.4** – Analog sensor alert — NTC temperature above high threshold*

Test 3: Digital Sensor Alert Trigger

Similarly, when the DS18B20 temperature rises above 30°C (adjusted via the Wokwi temperature slider), the digital channel's ThresholdAlert FSM triggers independently. The yellow LED activates to indicate the digital alert, while the red LED state depends only on the analog channel.

[Screenshot pending — set DS18B20 slider above 30°C in Wokwi.
Yellow LED on, Green LED off, Red LED off.
LCD showing "A:25.0C OK D:33.0C !!".
Save to resources/ObtainedResults/test_digital_alert.png]

***Figure 3.5** – Digital sensor alert — DS18B20 temperature above high threshold*

Test 4: Dual Sensor Alert

When both sensors simultaneously exceed the 30°C threshold, both the red and yellow LEDs activate, the green LED turns off, and the LCD shows "!!" status for both channels. This test verifies that the two ThresholdAlert instances operate independently and correctly.

[Screenshot pending — set both NTC and DS18B20 above 30°C.
Red LED on, Yellow LED on, Green LED off.
LCD showing "A:32.5C !! D:33.0C !!".
Save to resources/ObtainedResults/test_dual_alert.png]

***Figure 3.6** – Dual sensor alert — both sensors above high threshold*

Test 5: Hysteresis Behavior

The hysteresis test demonstrates the dead-band between the high threshold (30°C) and the low threshold (28°C). Once an alert is active, the temperature must drop below 28°C (not just below 30°C) for the alert to clear. This prevents rapid toggling when the temperature oscillates near a single threshold value.

The test procedure is as follows:

1. Start with the NTC temperature at 25°C (NORMAL state).
2. Increase the temperature to 31°C → alert activates after debounce.
3. Decrease to 29°C → alert remains active (above T_{low} = 28°C).

4. Decrease to 27°C → alert clears after debounce.

[Screenshot pending — demonstrate hysteresis by lowering NTC from 31°C to 29°C (alert stays) then to 27°C (alert clears). Capture serial output showing state transitions. Save to resources/ObtainedResults/test_hysteresis.png]

***Figure 3.7** – Hysteresis behavior demonstration — alert persists in dead-band zone*

Test 6: Debounce Validation

The debounce test validates that brief transient spikes do not trigger false alerts. By quickly adjusting the NTC temperature above and below 30°C (before 5 consecutive readings are accumulated), the system should return to NORMAL without entering ALERT_ACTIVE.

[Screenshot pending — briefly raise NTC above 30°C then lower before 5 confirmations. Serial shows DB_HIGH → NORMAL. Save to resources/ObtainedResults/test_debounce.png]

***Figure 3.8** – Debounce validation — brief spike rejected by debounce filter*

3.4. Serial Monitor Output

The structured STDIO report generated every 2 seconds provides comprehensive diagnostic information. A typical report output includes raw ADC values, computed resistance, both temperatures, alert FSM states, debounce counters, cumulative statistics, and system uptime.

[Screenshot pending — capture the serial monitor showing a full structured STDIO report with all fields visible. Save to resources/ObtainedResults/test_report.png]

***Figure 3.9** – Structured STDIO diagnostic report from serial monitor*

The report includes the following fields:

- **Sensor Readings**: Raw ADC value (0–1023), computed NTC resistance (Ω), analog temperature (°C), digital temperature (°C), and validity flags.
- **Alert Status**: Current FSM state (NORMAL, DB_HIGH, ALERT, DB_LOW), debounce counter progress (e.g., "3/5"), and filtered temperature value for both channels.
- **Statistics**: Total readings count, analog and digital alert activation counts, conditioning cycles, and system uptime in seconds.

3.5. LCD Display Pages

The LCD display alternates between two information pages every 500 ms:

- **Page 0** shows the current temperature readings and alert status:

```
A:25.3C OK
D:24.8C OK
```

When an alert is active, the status changes from "OK" to "!!".

- **Page 1** shows the threshold configuration:

```
TH:30.0/28.0C
Alerts: A:0 D:0
```

The first line displays the high and low thresholds, and the second line shows the cumulative alert count for each channel.

3.6. Verification Summary

The following table summarizes the test results for all scenarios:

Test Scenario	Result	Observation
Normal operation	PASS	Green LED on, no alerts
Analog alert trigger	PASS	Red LED on after debounce
Digital alert trigger	PASS	Yellow LED on after debounce
Dual sensor alert	PASS	Both alert LEDs active
Hysteresis behavior	PASS	Alert persists in dead-band
Debounce (false spike)	PASS	Transient rejected
LCD page alternation	PASS	Both pages display correctly
STDIO report format	PASS	All fields populated

Table 2: Test verification summary for Lab 3.1

All test scenarios produced the expected behavior, confirming that the dual-sensor monitoring system correctly implements the signal acquisition, conditioning, and display pipeline as designed.

4. Conclusions

4.1. Achievement of Objectives

This laboratory work successfully implemented a dual-sensor temperature monitoring system that fulfills all requirements specified in Variant C of Lab 3.1. The system monitors two independent temperature sensors — an analog NTC thermistor and a digital DS18B20 — through a three-stage FreeRTOS pipeline consisting of acquisition, signal conditioning, and display tasks.

The primary objectives achieved include:

- **Dual-sensor data acquisition**: Both analog (ADC-based) and digital (OneWire-based) temperature sensors are read concurrently within a single FreeRTOS task at a consistent 50 ms sampling rate, demonstrating the integration of fundamentally different sensor interfaces within a unified acquisition framework.

- **Signal conditioning with hysteresis and debounce**: The ThresholdAlert module implements a robust 4-state FSM that prevents both noise-induced false alarms (through debounce filtering) and threshold oscillation (through hysteresis with a 2°C dead-band between the 30°C high threshold and 28°C low threshold).

- **Real-time visual and textual feedback**: The system provides three output channels — LED indicators for immediate visual status, an LCD display for formatted temperature readings, and structured STDIO reports for detailed diagnostic information — ensuring comprehensive observability at multiple levels of detail.

- **FreeRTOS concurrency**: The three tasks operate at different priorities and timing modes (periodic acquisition, event-driven conditioning, periodic display), demonstrating proper use of FreeRTOS primitives including binary semaphores, mutexes with priority inheritance, and `vTaskDelayUntil()` for drift-free periodic execution.

4.2. Performance Analysis

The system operates within comfortable resource margins on the ATmega2560 platform. Flash usage of 10.3% (26 122 bytes) leaves substantial space for additional features such as data logging, calibration storage, or more complex display modes. RAM usage of 24.0% (1 969 bytes) provides adequate headroom for the three FreeRTOS task stacks and the shared data structures, though care must be taken when adding features that increase stack depth (recursive algorithms, large local buffers).

The 50 ms acquisition period provides a 20 Hz sampling rate that is well-suited for temperature monitoring applications, where thermal time constants are typically on the order of seconds to minutes. The DS18B20's non-blocking conversion approach ensures that the slower digital sensor does not introduce timing jitter into the acquisition loop — the 187.5 ms conversion time at 10-bit resolution is fully hidden behind three consecutive 50 ms sleep periods.

The mutex-based access to shared data structures introduces minimal overhead since contention is rare: Task 2 holds the mutex only briefly to copy data, and Task 3's lower priority ensures it does not preempt the higher-priority tasks during critical sections. The binary semaphore provides efficient event-driven wakeup of Task 2, avoiding the power waste and latency of polling approaches.

4.3. Limitations

Several limitations exist in the current implementation:

- **Fixed thresholds**: The high (30°C) and low (28°C) thresholds are compile-time constants. A production system would benefit from runtime-configurable thresholds via a serial command interface or keypad input.
- **No data logging**: The system reports current values only and does not store historical data. Adding a circular buffer or SD card logging would enable trend analysis and event correlation.
- **NTC accuracy**: The Steinhart-Hart Beta equation is a simplification that provides approximately ±1°C accuracy across a moderate temperature range. Higher accuracy would require a full three-coefficient Steinhart-Hart model with individual thermistor calibration.
- **Single-sensor assumption**: The DigitalTempSensor module reads only the first DS18B20 on the bus. The OneWire protocol supports multiple devices, and extending the module to handle device addressing would enable monitoring of additional sensors on the same data line.
- **LCD update rate**: The 500 ms LCD refresh period and 2-page alternation scheme provide adequate feedback for manual observation but may miss brief transient events. A triggered display mode that immediately shows alert transitions would improve responsiveness.

4.4. Proposed Improvements

Future enhancements could extend the system in several directions:

- **Moving average filter**: Applying a simple moving average or exponential weighted moving average (EWMA) filter to the raw sensor readings would further smooth noise without adding the latency of the debounce mechanism.
- **Adaptive thresholds**: Implementing auto-calibration that adjusts thresholds based on observed temperature baseline during a startup learning phase would make the system more portable across different environments.
- **Remote monitoring**: Adding an Ethernet or Wi-Fi module (e.g., ESP8266) would enable MQTT publishing of sensor data to a cloud dashboard, extending the system from a local monitor to an IoT edge device.

- **Audible alarm**: Adding a passive buzzer with PWM-driven tone generation would provide an audible alert complement to the visual LED indicators, improving the alert's visibility in noisy or visually cluttered environments.

- **Multi-sensor expansion**: Leveraging the OneWire bus capability to add multiple DS18B20 sensors at different physical locations would create a distributed temperature monitoring network using a single data pin.

4.5. Real-World Impact

Temperature monitoring with dual-sensor redundancy and signal conditioning is widely applicable in industrial and commercial contexts. Server room environmental monitoring systems use similar architectures to detect overheating conditions before equipment damage occurs, with the hysteresis mechanism preventing alarm fatigue from threshold oscillation near air conditioning setpoints. Agricultural greenhouse control systems monitor multiple temperature zones with NTC thermistors (for their low cost and wide availability) alongside calibrated digital sensors (for their higher accuracy), using the readings from both to cross-validate measurements and trigger ventilation or heating systems. Medical cold chain monitoring — particularly relevant during vaccine transportation — requires reliable temperature alerting with no false alarms, making the debounce-filtered hysteresis approach a natural fit for ensuring that only sustained temperature excursions trigger corrective actions.

The modular software architecture developed in this lab, with its clean separation between sensor drivers, signal conditioning logic, and application-level tasks, mirrors the design patterns used in professional embedded systems frameworks such as AUTOSAR and the ARM CMSIS ecosystem. The experience gained in designing reusable library modules, implementing FreeRTOS-based concurrency, and applying signal conditioning techniques provides a foundation directly applicable to industrial embedded systems development.

5. Note Regarding Usage of AI

During the preparation of this laboratory report, the author utilized artificial intelligence tools to enhance the quality and presentation of the documentation. Specifically, the following AI tools were employed:

- **Report Formatting:** Assistance with LaTeX document structure, formatting consistency, and adherence to academic writing standards.

- **Report Template Generation:** Creation of the initial document template with proper sectioning and organizational structure.

- **Content Rephrasing:** Refinement of technical descriptions and explanations to ensure grammatical accuracy and clarity without altering the technical substance.

- **Code Documentation:** Generation of inline comments within the source code to improve code readability and maintainability.
- **Development Environment Setup:** Configuration assistance for PlatformIO integration with the Wokwi simulation platform.

All AI-generated content was thoroughly reviewed, validated, and adjusted by the author to ensure technical accuracy, alignment with project requirements, and adherence to embedded systems best practices. The core technical implementation, circuit design, system architecture decisions, and experimental results represent the author's original work and understanding of the subject matter. The AI tools served as assistive technologies to enhance documentation quality and development workflow efficiency, while the author retained full responsibility for the content, correctness, and conclusions presented in this report.

6. Bibliography

[1] Bragarenco, A., Astafi, V., *Sisteme Electronice Încorporate: Indicații metodice pentru lucrări de laborator*, Universitatea Tehnică a Moldovei, Chișinău, 2024.

[2] Technical University of Moldova, *Embedded Systems: Practical Guide for Hardware-Software Interface Development*, Department of Microelectronics and Biomedical Engineering, Chișinău, 2024.

[3] Maxim Integrated, *DS18B20 Programmable Resolution 1-Wire Digital Thermometer*, Datasheet Rev. 042208, 2019. Available: `https://datasheets.maximintegrated.com/en/ds/DS18B20.pdf`

[4] Murata Manufacturing Co., *NTC Thermistors — NTC Thermistor Technical Note*, Application Note, 2021. Available: `https://www.murata.com/en-us/support/faqs/products/thermistor/ntc`

[5] Richard Barry, *Mastering the FreeRTOS Real Time Kernel — A Hands-On Tutorial Guide*, Real Time Engineers Ltd., 2016. Available: `https://www.freertos.org/Documentation/RTOS_book.html`

[6] Arduino, *Arduino Mega 2560 Rev3 Documentation*, `https://docs.arduino.cc/hardware/mega-2560/`, Accessed: 2025-04-10.

[7] PlatformIO, *Professional collaborative platform for embedded development*, `https://platformio.org/`, Accessed: 2025-04-10.

[8] Wokwi, *Online Electronics Simulator — Arduino, ESP32, Raspberry Pi Pico*, `https://wokwi.com/`, Accessed: 2025-04-10.

[9] Miles Burton, *DallasTemperature Arduino Library*, `https://github.com/milesburton/Arduino-Temperature-Control-Library`, Accessed: 2025-04-10.

[10] Paul Stoffregen, *OneWire Arduino Library*, `https://github.com/PaulStoffregen/OneWire`, Accessed: 2025-04-10.

7. Appendix — Source Code

The complete source code for Laboratory Work 3.1 is organized below by architectural layer. All source files are available in the project GitHub repository.

GitHub Repository: `https://github.com/avremere/es-labs`

7.1. Driver Layer: AnalogTempSensor

AnalogTempSensor.h

Listing 7.1 *– AnalogTempSensor.h — NTC Thermistor Driver Interface*

```
/**
 * @file AnalogTempSensor.h
 * @brief NTC Thermistor Analog Temperature Sensor Driver Interface
 *
 * Provides a reusable abstraction for reading temperature from an NTC
 * thermistor connected to an analog ADC pin through a voltage divider
 * circuit. Converts raw ADC readings to temperature in degrees Celsius
 * using the Steinhart-Hart Beta parameter equation.
 *
 * Circuit configuration (voltage divider):
 *   VCC --- [R_series] --- ADC_PIN --- [NTC] --- GND
 *
 * The NTC resistance is calculated from the ADC reading and then
 * converted to temperature using:
 *   1/T = 1/T0 + (1/B) * ln(R_ntc / R0)
 *
 * Usage:
 *   AnalogTempSensor ntc(A0, 10000, 10000, 3950);
 *   ntc.init();
 *   float tempC = ntc.readTemperatureC();
 */

#ifndef ANALOG_TEMP_SENSOR_H
#define ANALOG_TEMP_SENSOR_H

#include <Arduino.h>

class AnalogTempSensor {
public:
```

```
    AnalogTempSensor(uint8_t adcPin, uint32_t seriesR,
                     uint32_t nominalR, uint16_t betaCoeff,
                     float nominalTempC = 25.0f,
                     uint16_t adcResolution = 10);
    void init();
    uint16_t readRaw();
    float readResistance();
    float readTemperatureC();
    float readTemperatureF();
    bool isValid() const;
    uint16_t getLastRaw() const;
    float getLastTemperatureC() const;

private:
    uint8_t  _adcPin;
    uint32_t _seriesR;
    uint32_t _nominalR;
    uint16_t _betaCoeff;
    float    _nominalTempK;
    uint16_t _adcMax;
    uint16_t _lastRaw;
    float    _lastTempC;
    bool     _valid;
};

#endif // ANALOG_TEMP_SENSOR_H
```

AnalogTempSensor.cpp

Listing 7.2 *– AnalogTempSensor.cpp — NTC Thermistor Driver Implementation*

```
/**
 * @file AnalogTempSensor.cpp
 * @brief NTC Thermistor Analog Temperature Sensor Driver Implementation
 *
 * Implements the ADC-based temperature reading for an NTC thermistor
 * connected through a voltage divider circuit. Uses the Steinhart-Hart
 * Beta parameter equation for resistance-to-temperature conversion.
 */

#include "AnalogTempSensor.h"
#include <math.h>

// Constructor
AnalogTempSensor::AnalogTempSensor(uint8_t adcPin, uint32_t seriesR,
                                   uint32_t nominalR, uint16_t betaCoeff,
                                   float nominalTempC, uint16_t adcResolution)
    : _adcPin(adcPin),
```

```
      _seriesR(seriesR),
      _nominalR(nominalR),
      _betaCoeff(betaCoeff),
      _nominalTempK(nominalTempC + 273.15f),
      _adcMax((1 << adcResolution) - 1),
      _lastRaw(0),
      _lastTempC(NAN),
      _valid(false) {}

void AnalogTempSensor::init() {
    pinMode(_adcPin, INPUT);
    _lastRaw  = 0;
    _lastTempC = NAN;
    _valid    = false;
}

uint16_t AnalogTempSensor::readRaw() {
    _lastRaw = analogRead(_adcPin);
    return _lastRaw;
}

float AnalogTempSensor::readResistance() {
    uint16_t adc = readRaw();

    // Guard against division by zero or open/short circuit.
    if (adc == 0 || adc >= _adcMax) {
        _valid = false;
        return -1.0f;
    }

    // Voltage divider: R_ntc = R_series * ADC / (ADC_MAX - ADC)
    float resistance = (float)_seriesR * (float)adc
                       / (float)(_adcMax - adc);
    _valid = true;
    return resistance;
}

float AnalogTempSensor::readTemperatureC() {
    float resistance = readResistance();
    if (resistance < 0.0f) {
        _lastTempC = NAN;
        _valid = false;
        return NAN;
    }

    // Steinhart-Hart Beta equation:
    // 1/T = 1/T0 + (1/B) * ln(R / R0)
    float steinhart = log(resistance / (float)_nominalR);
```

```
    steinhart /= (float)_betaCoeff;
    steinhart += 1.0f / _nominalTempK;
    float tempK = 1.0f / steinhart;
    _lastTempC = tempK - 273.15f;
    _valid = true;
    return _lastTempC;
}

float AnalogTempSensor::readTemperatureF() {
    float tempC = readTemperatureC();
    if (isnan(tempC)) return NAN;
    return tempC * 9.0f / 5.0f + 32.0f;
}

bool AnalogTempSensor::isValid() const {
    return _valid;
}

uint16_t AnalogTempSensor::getLastRaw() const {
    return _lastRaw;
}

float AnalogTempSensor::getLastTemperatureC() const {
    return _lastTempC;
}
```

7.2. Driver Layer: DigitalTempSensor

DigitalTempSensor.h

Listing 7.3 *– DigitalTempSensor.h — DS18B20 OneWire Driver Interface*

```
/**
 * @file DigitalTempSensor.h
 * @brief DS18B20 Digital Temperature Sensor Driver Interface (OneWire)
 *
 * Provides a reusable abstraction for reading temperature from a Dallas
 * DS18B20 digital temperature sensor using the OneWire protocol.
 *
 * Circuit configuration:
 *   VCC (5V) --+-- DS18B20 VDD
 *              |
 *           [4.7K]  (pull-up resistor)
 *              |
 *   DATA_PIN --+-- DS18B20 DQ
 *   GND -------+-- DS18B20 GND
 */
```

```
#ifndef DIGITAL_TEMP_SENSOR_H
#define DIGITAL_TEMP_SENSOR_H

#include <Arduino.h>
#include <OneWire.h>
#include <DallasTemperature.h>

class DigitalTempSensor {
public:
    DigitalTempSensor(uint8_t dataPin,
                      uint8_t resolution = 10);
    bool init();
    void requestConversion();
    bool isConversionComplete();
    float readTemperatureC();
    float getLastTemperatureC() const;
    float readTemperatureF();
    bool isConnected() const;
    bool isValid() const;
    uint8_t getDeviceCount() const;

private:
    OneWire           _oneWire;
    DallasTemperature _sensors;
    uint8_t           _resolution;
    float             _lastTempC;
    bool              _connected;
    bool              _valid;
    uint8_t           _deviceCount;
};

#endif // DIGITAL_TEMP_SENSOR_H
```

DigitalTempSensor.cpp

Listing 7.4 *– DigitalTempSensor.cpp — DS18B20 OneWire Driver Implementation*

```
/**
 * @file DigitalTempSensor.cpp
 * @brief DS18B20 Digital Temperature Sensor Driver Implementation
 *
 * Implements OneWire-based temperature reading from the DS18B20 sensor.
 * Uses the DallasTemperature library for protocol handling and provides
 * both blocking and non-blocking read modes.
 */

#include "DigitalTempSensor.h"
```

```
static const float DISCONNECTED_TEMP = -127.0f;
static const float POWER_ON_RESET_TEMP = 85.0f;

DigitalTempSensor::DigitalTempSensor(uint8_t dataPin,
                                     uint8_t resolution)
    : _oneWire(dataPin),
      _sensors(&_oneWire),
      _resolution(resolution),
      _lastTempC(NAN),
      _connected(false),
      _valid(false),
      _deviceCount(0) {}

bool DigitalTempSensor::init() {
    _sensors.begin();
    _deviceCount = _sensors.getDeviceCount();
    if (_deviceCount == 0) {
        _connected = false;
        return false;
    }
    _connected = true;
    _sensors.setResolution(_resolution);
    _sensors.setWaitForConversion(false);
    return true;
}

void DigitalTempSensor::requestConversion() {
    if (_connected) {
        _sensors.requestTemperatures();
    }
}

bool DigitalTempSensor::isConversionComplete() {
    if (!_connected) return false;
    return _sensors.isConversionComplete();
}

float DigitalTempSensor::readTemperatureC() {
    if (!_connected) {
        _valid = false;
        _lastTempC = NAN;
        return NAN;
    }
    _sensors.setWaitForConversion(true);
    _sensors.requestTemperatures();
    _sensors.setWaitForConversion(false);

    float tempC = _sensors.getTempCByIndex(0);
```

```
    if (tempC == DEVICE_DISCONNECTED_C ||
        tempC <= DISCONNECTED_TEMP) {
        _valid = false;
        _lastTempC = NAN;
        return NAN;
    }
    if (tempC == POWER_ON_RESET_TEMP) {
        _valid = false;
        return _lastTempC;
    }
    _valid = true;
    _lastTempC = tempC;
    return _lastTempC;
}

float DigitalTempSensor::getLastTemperatureC() const {
    return _lastTempC;
}

float DigitalTempSensor::readTemperatureF() {
    float tempC = readTemperatureC();
    if (isnan(tempC)) return NAN;
    return tempC * 9.0f / 5.0f + 32.0f;
}

bool DigitalTempSensor::isConnected() const {
    return _connected;
}

bool DigitalTempSensor::isValid() const {
    return _valid;
}

uint8_t DigitalTempSensor::getDeviceCount() const {
    return _deviceCount;
}
```

7.3. Service Layer: ThresholdAlert

ThresholdAlert.h

Listing 7.5 – ThresholdAlert.h — Hysteresis-Based Threshold Alert Interface

```
/**
 * @file ThresholdAlert.h
 * @brief Hysteresis-Based Threshold Alert Module Interface
 *
```

```
 * 4-state FSM: NORMAL -> DEBOUNCE_HIGH -> ALERT -> DEBOUNCE_LOW -> NORMAL
 * Each transition requires N consecutive confirmations (debounce).
 * Hysteresis band: [lowThreshold, highThreshold]
 */

#ifndef THRESHOLD_ALERT_H
#define THRESHOLD_ALERT_H

#include <Arduino.h>

enum AlertState {
    ALERT_NORMAL,
    ALERT_DEBOUNCE_HIGH,
    ALERT_ACTIVE,
    ALERT_DEBOUNCE_LOW
};

class ThresholdAlert {
public:
    ThresholdAlert(float highThreshold, float lowThreshold,
                   uint8_t debounceCount = 5);
    void init();
    AlertState update(float value);
    AlertState getState() const;
    bool isAlertActive() const;
    bool isDebouncing() const;
    uint8_t getDebounceCounter() const;
    const char* getStateString() const;
    void setThresholds(float high, float low);
    void setDebounceCount(uint8_t count);
    float getHighThreshold() const;
    float getLowThreshold() const;

private:
    float      _highThreshold;
    float      _lowThreshold;
    uint8_t    _debounceMax;
    uint8_t    _debounceCounter;
    AlertState _state;
};

#endif // THRESHOLD_ALERT_H
```

ThresholdAlert.cpp

Listing 7.6 *– ThresholdAlert.cpp — Hysteresis-Based Threshold Alert Implementation*

```
/**
```

```
 * @file ThresholdAlert.cpp
 * @brief Hysteresis-Based Threshold Alert Module Implementation
 *
 * Implements the 4-state FSM for threshold detection with hysteresis
 * and debounce filtering.
 */

#include "ThresholdAlert.h"

static const char* STATE_NAMES[] = {
    "NORMAL", "DEBOUNCE_HIGH", "ALERT", "DEBOUNCE_LOW"
};

ThresholdAlert::ThresholdAlert(float highThreshold,
                               float lowThreshold,
                               uint8_t debounceCount)
    : _highThreshold(highThreshold),
      _lowThreshold(lowThreshold),
      _debounceMax(debounceCount),
      _debounceCounter(0),
      _state(ALERT_NORMAL) {}

void ThresholdAlert::init() {
    _state = ALERT_NORMAL;
    _debounceCounter = 0;
}

AlertState ThresholdAlert::update(float value) {
    switch (_state) {
    case ALERT_NORMAL:
        if (value > _highThreshold) {
            _state = ALERT_DEBOUNCE_HIGH;
            _debounceCounter = 1;
        }
        break;
    case ALERT_DEBOUNCE_HIGH:
        if (value > _highThreshold) {
            _debounceCounter++;
            if (_debounceCounter >= _debounceMax) {
                _state = ALERT_ACTIVE;
                _debounceCounter = 0;
            }
        } else {
            _state = ALERT_NORMAL;
            _debounceCounter = 0;
        }
        break;
    case ALERT_ACTIVE:
```

```
        if (value < _lowThreshold) {
            _state = ALERT_DEBOUNCE_LOW;
            _debounceCounter = 1;
        }
        break;
    case ALERT_DEBOUNCE_LOW:
        if (value < _lowThreshold) {
            _debounceCounter++;
            if (_debounceCounter >= _debounceMax) {
                _state = ALERT_NORMAL;
                _debounceCounter = 0;
            }
        } else {
            _state = ALERT_ACTIVE;
            _debounceCounter = 0;
        }
        break;
    }
    return _state;
}

AlertState ThresholdAlert::getState() const { return _state; }

bool ThresholdAlert::isAlertActive() const {
    return _state == ALERT_ACTIVE;
}

bool ThresholdAlert::isDebouncing() const {
    return (_state == ALERT_DEBOUNCE_HIGH) ||
           (_state == ALERT_DEBOUNCE_LOW);
}

uint8_t ThresholdAlert::getDebounceCounter() const {
    return _debounceCounter;
}

const char* ThresholdAlert::getStateString() const {
    return STATE_NAMES[_state];
}

void ThresholdAlert::setThresholds(float highThreshold,
                                   float lowThreshold) {
    _highThreshold = highThreshold;
    _lowThreshold  = lowThreshold;
    _state = ALERT_NORMAL;
    _debounceCounter = 0;
}
```

```
void ThresholdAlert::setDebounceCount(uint8_t count) {
    _debounceMax = count;
    _debounceCounter = 0;
}

float ThresholdAlert::getHighThreshold() const {
    return _highThreshold;
}

float ThresholdAlert::getLowThreshold() const {
    return _lowThreshold;
}
```

7.4. Application Layer: Shared Data and Synchronization

sensor_data.h

Listing 7.7 – sensor_data.h — Shared Data Structures and Pin Mapping

```
/**
 * @file sensor_data.h
 * @brief Lab 3.1 -- Shared Data Structures and Synchronization Primitives
 *
 * Declares the inter-task shared data structures, FreeRTOS synchronization
 * handles, hardware pin mapping, and application constants.
 */

#ifndef SENSOR_DATA_H
#define SENSOR_DATA_H

#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <semphr.h>
#include "ThresholdAlert.h"

// === Hardware Pin Mapping (Arduino Mega 2560) ===
static const uint8_t PIN_ANALOG_SENSOR  = A0;
static const uint8_t PIN_DIGITAL_SENSOR = 2;
static const uint8_t PIN_LED_GREEN      = 8;
static const uint8_t PIN_LED_RED        = 9;
static const uint8_t PIN_LED_YELLOW     = 10;
static const uint8_t LCD_I2C_ADDRESS    = 0x27;
static const uint8_t LCD_COLS           = 16;
static const uint8_t LCD_ROWS           = 2;

// === NTC Thermistor Parameters ===
static const uint32_t NTC_SERIES_RESISTANCE  = 10000;
static const uint32_t NTC_NOMINAL_RESISTANCE = 10000;
```

```
static const uint16_t NTC_BETA_COEFFICIENT   = 3950;
static const float    NTC_NOMINAL_TEMP_C     = 25.0f;

// === DS18B20 Parameters ===
static const uint8_t DS18B20_RESOLUTION = 10;

// === Threshold Alert Parameters ===
static const float   ANALOG_THRESHOLD_HIGH  = 30.0f;
static const float   ANALOG_THRESHOLD_LOW   = 28.0f;
static const float   DIGITAL_THRESHOLD_HIGH = 30.0f;
static const float   DIGITAL_THRESHOLD_LOW  = 28.0f;
static const uint8_t ALERT_DEBOUNCE_COUNT   = 5;

// === FreeRTOS Task Configuration ===
static const uint32_t TASK_ACQUISITION_PERIOD_MS  = 50;
static const UBaseType_t TASK_ACQUISITION_PRIORITY = 3;
static const configSTACK_DEPTH_TYPE TASK_ACQUISITION_STACK = 256;

static const uint32_t TASK_CONDITIONING_PERIOD_MS  = 100;
static const UBaseType_t TASK_CONDITIONING_PRIORITY = 2;
static const configSTACK_DEPTH_TYPE TASK_CONDITIONING_STACK = 256;

static const uint32_t TASK_DISPLAY_PERIOD_MS  = 500;
static const UBaseType_t TASK_DISPLAY_PRIORITY = 1;
static const configSTACK_DEPTH_TYPE TASK_DISPLAY_STACK = 512;

// === Shared Data Structures ===
typedef struct {
    uint16_t   analogRaw;
    float      analogResistance;
    float      analogTempC;
    bool       analogValid;
    float      digitalTempC;
    bool       digitalValid;
    uint32_t   readingCount;
    TickType_t timestamp;
} SensorReadings_t;

typedef struct {
    AlertState analogAlertState;
    uint8_t    analogDebounceCount;
    float      analogFilteredTemp;
    AlertState digitalAlertState;
    uint8_t    digitalDebounceCount;
    float      digitalFilteredTemp;
    uint32_t   analogAlertCount;
    uint32_t   digitalAlertCount;
    uint32_t   conditioningCycles;
```

```
} AlertStatus_t;

// === External Declarations ===
extern SensorReadings_t g_sensorData;
extern AlertStatus_t    g_alertData;
extern SemaphoreHandle_t xNewReadingSemaphore;
extern SemaphoreHandle_t xSensorMutex;

void sensorDataInit();

#endif // SENSOR_DATA_H
```

sensor_data.cpp

Listing 7.8 *– sensor_data.cpp — Shared State Definitions*

```
/**
 * @file sensor_data.cpp
 * @brief Lab 3.1 -- Shared State Definitions and Synchronization Init
 */

#include "sensor_data.h"

SensorReadings_t g_sensorData = {
    0, 0.0f, 0.0f, false, 0.0f, false, 0, 0
};

AlertStatus_t g_alertData = {
    ALERT_NORMAL, 0, 0.0f,
    ALERT_NORMAL, 0, 0.0f,
    0, 0, 0
};

SemaphoreHandle_t xNewReadingSemaphore = NULL;
SemaphoreHandle_t xSensorMutex         = NULL;

void sensorDataInit() {
    xNewReadingSemaphore = xSemaphoreCreateBinary();
    xSensorMutex = xSemaphoreCreateMutex();
}
```

7.5. Application Layer: Task 1 — Sensor Acquisition

task_acquisition.h

Listing 7.9 *– task_acquisition.h — Sensor Acquisition Task Interface*

```
/**
 * @file task_acquisition.h
 * @brief Lab 3.1 -- Sensor Acquisition Task Interface (Task 1)
 *
 * Periodically reads both the analog NTC thermistor (via ADC) and
 * the digital DS18B20 sensor (via OneWire) at 50 ms intervals.
 */

#ifndef TASK_ACQUISITION_H
#define TASK_ACQUISITION_H

#include <Arduino_FreeRTOS.h>

void vTaskAcquisition(void *pvParameters);

#endif // TASK_ACQUISITION_H
```

task_acquisition.cpp

Listing 7.10 – *task_acquisition.cpp — Sensor Acquisition Task Implementation*

```
/**
 * @file task_acquisition.cpp
 * @brief Lab 3.1 -- Sensor Acquisition Task Implementation (Task 1)
 *
 * Reads NTC thermistor via ADC and DS18B20 via OneWire at 50 ms
 * intervals using vTaskDelayUntil() for drift-free timing.
 */

#include "task_acquisition.h"
#include "sensor_data.h"
#include "AnalogTempSensor.h"
#include "DigitalTempSensor.h"

static AnalogTempSensor s_ntcSensor(
    PIN_ANALOG_SENSOR, NTC_SERIES_RESISTANCE,
    NTC_NOMINAL_RESISTANCE, NTC_BETA_COEFFICIENT,
    NTC_NOMINAL_TEMP_C);

static DigitalTempSensor s_ds18b20(
    PIN_DIGITAL_SENSOR, DS18B20_RESOLUTION);

void vTaskAcquisition(void *pvParameters) {
    (void)pvParameters;

    s_ntcSensor.init();
    bool ds18b20Found = s_ds18b20.init();
```

```
    if (ds18b20Found) {
        s_ds18b20.requestConversion();
    }

    bool firstConversion = true;
    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xPeriod =
        pdMS_TO_TICKS(TASK_ACQUISITION_PERIOD_MS);

    uint16_t rawAdc;
    float resistance, analogTemp, digitalTemp;
    bool analogOk, digitalOk;

    for (;;) {
        vTaskDelayUntil(&xLastWakeTime, xPeriod);

        // Read analog sensor (NTC thermistor)
        analogTemp = s_ntcSensor.readTemperatureC();
        rawAdc     = s_ntcSensor.getLastRaw();
        resistance = s_ntcSensor.readResistance();
        analogOk   = s_ntcSensor.isValid();

        // Read digital sensor (DS18B20)
        if (ds18b20Found) {
            if (firstConversion) {
                while (!s_ds18b20.isConversionComplete()) {
                    vTaskDelay(pdMS_TO_TICKS(10));
                }
                firstConversion = false;
            }
            if (s_ds18b20.isConversionComplete()) {
                digitalTemp = s_ds18b20.readTemperatureC();
                digitalOk   = s_ds18b20.isValid();
            } else {
                digitalTemp = s_ds18b20.getLastTemperatureC();
                digitalOk   = s_ds18b20.isValid();
            }
            s_ds18b20.requestConversion();
        } else {
            digitalTemp = NAN;
            digitalOk   = false;
        }

        // Write to shared data under mutex
        if (xSemaphoreTake(xSensorMutex,
                           pdMS_TO_TICKS(10)) == pdTRUE) {
            g_sensorData.analogRaw        = rawAdc;
            g_sensorData.analogResistance = resistance;
```

```
            g_sensorData.analogTempC      = analogTemp;
            g_sensorData.analogValid      = analogOk;
            g_sensorData.digitalTempC     = digitalTemp;
            g_sensorData.digitalValid     = digitalOk;
            g_sensorData.readingCount++;
            g_sensorData.timestamp = xTaskGetTickCount();
            xSemaphoreGive(xSensorMutex);
        }

        // Notify Task 2
        xSemaphoreGive(xNewReadingSemaphore);
    }
}
```

7.6. Application Layer: Task 2 — Signal Conditioning

task_conditioning.h

Listing 7.11 *– task_conditioning.h — Signal Conditioning Task Interface*

```
/**
 * @file task_conditioning.h
 * @brief Lab 3.1 -- Signal Conditioning & Alert Task Interface (Task 2)
 *
 * Event-driven task: wakes on semaphore from Task 1, applies
 * hysteresis-based threshold detection and controls alert LEDs.
 */

#ifndef TASK_CONDITIONING_H
#define TASK_CONDITIONING_H

#include <Arduino_FreeRTOS.h>

void vTaskConditioning(void *pvParameters);

#endif // TASK_CONDITIONING_H
```

task_conditioning.cpp

Listing 7.12 *– task_conditioning.cpp — Signal Conditioning Task Implementation*

```
/**
 * @file task_conditioning.cpp
 * @brief Lab 3.1 -- Signal Conditioning & Alert Task (Task 2)
 *
 * Applies threshold detection with hysteresis and debounce to both
 * analog and digital temperature readings. Controls LED indicators.
 */
```

```
#include "task_conditioning.h"
#include "sensor_data.h"
#include "ThresholdAlert.h"
#include "Led.h"

static ThresholdAlert s_analogAlert(
    ANALOG_THRESHOLD_HIGH, ANALOG_THRESHOLD_LOW,
    ALERT_DEBOUNCE_COUNT);

static ThresholdAlert s_digitalAlert(
    DIGITAL_THRESHOLD_HIGH, DIGITAL_THRESHOLD_LOW,
    ALERT_DEBOUNCE_COUNT);

static Led s_greenLed(PIN_LED_GREEN);
static Led s_redLed(PIN_LED_RED);
static Led s_yellowLed(PIN_LED_YELLOW);

void vTaskConditioning(void *pvParameters) {
    (void)pvParameters;

    s_analogAlert.init();
    s_digitalAlert.init();
    s_greenLed.init();
    s_redLed.init();
    s_yellowLed.init();

    s_greenLed.turnOn();
    s_redLed.turnOff();
    s_yellowLed.turnOff();

    float analogTemp, digitalTemp;
    bool analogValid, digitalValid;
    AlertState prevAnalogState  = ALERT_NORMAL;
    AlertState prevDigitalState = ALERT_NORMAL;

    for (;;) {
        // Wait for new sensor data
        if (xSemaphoreTake(xNewReadingSemaphore,
                           portMAX_DELAY) != pdTRUE) {
            continue;
        }

        // Read sensor data under mutex
        if (xSemaphoreTake(xSensorMutex,
                           pdMS_TO_TICKS(10)) == pdTRUE) {
            analogTemp   = g_sensorData.analogTempC;
            digitalTemp  = g_sensorData.digitalTempC;
```

```
            analogValid  = g_sensorData.analogValid;
            digitalValid = g_sensorData.digitalValid;
            xSemaphoreGive(xSensorMutex);
        } else {
            continue;
        }

        // Apply threshold detection
        AlertState analogState  = ALERT_NORMAL;
        AlertState digitalState = ALERT_NORMAL;

        if (analogValid && !isnan(analogTemp)) {
            analogState = s_analogAlert.update(analogTemp);
        }
        if (digitalValid && !isnan(digitalTemp)) {
            digitalState = s_digitalAlert.update(digitalTemp);
        }

        // Write alert status under mutex
        if (xSemaphoreTake(xSensorMutex,
                           pdMS_TO_TICKS(10)) == pdTRUE) {
            g_alertData.analogAlertState    = analogState;
            g_alertData.analogDebounceCount =
                s_analogAlert.getDebounceCounter();
            g_alertData.analogFilteredTemp  = analogTemp;
            g_alertData.digitalAlertState    = digitalState;
            g_alertData.digitalDebounceCount =
                s_digitalAlert.getDebounceCounter();
            g_alertData.digitalFilteredTemp  = digitalTemp;

            if (analogState == ALERT_ACTIVE &&
                prevAnalogState != ALERT_ACTIVE) {
                g_alertData.analogAlertCount++;
            }
            if (digitalState == ALERT_ACTIVE &&
                prevDigitalState != ALERT_ACTIVE) {
                g_alertData.digitalAlertCount++;
            }
            g_alertData.conditioningCycles++;
            xSemaphoreGive(xSensorMutex);
        }

        prevAnalogState  = analogState;
        prevDigitalState = digitalState;

        // Update LED indicators
        bool anyAlert = (analogState == ALERT_ACTIVE) ||
                        (digitalState == ALERT_ACTIVE);
```

```
        if (anyAlert) s_greenLed.turnOff();
        else          s_greenLed.turnOn();

        if (analogState == ALERT_ACTIVE) {
            s_redLed.turnOn();
        } else if (analogState == ALERT_DEBOUNCE_HIGH ||
                   analogState == ALERT_DEBOUNCE_LOW) {
            s_redLed.toggle();  // Blink during debounce
        } else {
            s_redLed.turnOff();
        }

        if (digitalState == ALERT_ACTIVE) {
            s_yellowLed.turnOn();
        } else if (digitalState == ALERT_DEBOUNCE_HIGH ||
                   digitalState == ALERT_DEBOUNCE_LOW) {
            s_yellowLed.toggle();
        } else {
            s_yellowLed.turnOff();
        }
    }
}
```

7.7. Application Layer: Task 3 — Display and Reporting

task_display.h

Listing 7.13 *– task_display.h — Display and Reporting Task Interface*

```
/**
 * @file task_display.h
 * @brief Lab 3.1 -- Display & Reporting Task Interface (Task 3)
 *
 * Periodically updates the LCD display and prints structured
 * STDIO reports to the serial terminal at 500 ms intervals.
 */

#ifndef TASK_DISPLAY_H
#define TASK_DISPLAY_H

#include <Arduino_FreeRTOS.h>

void vTaskDisplay(void *pvParameters);

#endif // TASK_DISPLAY_H
```

task_display.cpp

Listing 7.14 – task_display.cpp — Display and Reporting Task Implementation

```
/**
 * @file task_display.cpp
 * @brief Lab 3.1 -- Display & Reporting Task Implementation (Task 3)
 *
 * Every 500 ms, reads sensor data and alert status, updates the LCD
 * with a summary, and prints a structured STDIO report every 2 seconds.
 */

#include "task_display.h"
#include "sensor_data.h"
#include "LcdDisplay.h"
#include <stdio.h>
#include <string.h>
#include <math.h>

static LcdDisplay s_lcd(LCD_I2C_ADDRESS, LCD_COLS, LCD_ROWS);

static void formatTemp(char *buf, size_t len, float temp) {
    if (isnan(temp)) strncpy(buf, " -- ", len);
    else             dtostrf(temp, 4, 1, buf);
}

static const char* alertFullLabel(AlertState state) {
    switch (state) {
        case ALERT_NORMAL:        return "NORMAL";
        case ALERT_DEBOUNCE_HIGH: return "DEBOUNCE_HIGH";
        case ALERT_ACTIVE:        return "ALERT";
        case ALERT_DEBOUNCE_LOW:  return "DEBOUNCE_LOW";
        default:                  return "UNKNOWN";
    }
}

void vTaskDisplay(void *pvParameters) {
    (void)pvParameters;

    s_lcd.init();
    s_lcd.backlight(true);
    s_lcd.showTwoLines("Lab 3.1 Sensors", "Initializing...");
    vTaskDelay(pdMS_TO_TICKS(1000));

    TickType_t xLastWakeTime = xTaskGetTickCount();
    const TickType_t xPeriod =
        pdMS_TO_TICKS(TASK_DISPLAY_PERIOD_MS);

    uint8_t displayCycle = 0;
```

```
    uint32_t reportNumber = 0;
    static const uint8_t REPORT_INTERVAL = 4;

    SensorReadings_t localSensor;
    AlertStatus_t    localAlert;
    char line0[17], line1[17];

    for (;;) {
        vTaskDelayUntil(&xLastWakeTime, xPeriod);
        displayCycle++;

        // Read shared data under mutex
        if (xSemaphoreTake(xSensorMutex,
                           pdMS_TO_TICKS(20)) == pdTRUE) {
            memcpy(&localSensor, &g_sensorData,
                   sizeof(SensorReadings_t));
            memcpy(&localAlert, &g_alertData,
                   sizeof(AlertStatus_t));
            xSemaphoreGive(xSensorMutex);
        } else {
            continue;
        }

        // Update LCD display
        uint8_t page = (displayCycle / 4) % 2;
        if (page == 0) {
            char aT[8], dT[8];
            formatTemp(aT, sizeof(aT),
                       localSensor.analogTempC);
            formatTemp(dT, sizeof(dT),
                       localSensor.digitalTempC);
            snprintf(line0, sizeof(line0),
                     "A:%sC D:%sC", aT, dT);

            bool aA = (localAlert.analogAlertState
                       == ALERT_ACTIVE);
            bool dA = (localAlert.digitalAlertState
                       == ALERT_ACTIVE);
            if (aA && dA)
                snprintf(line1, sizeof(line1), "ALERT: A+D");
            else if (aA)
                snprintf(line1, sizeof(line1), "ALERT: Analog");
            else if (dA)
                snprintf(line1, sizeof(line1), "ALERT: Digital");
            else
                snprintf(line1, sizeof(line1), "System: OK");
        } else {
            snprintf(line0, sizeof(line0), "AH:%.0f AL:%.0f C",
```

```
                    ANALOG_THRESHOLD_HIGH,
                    ANALOG_THRESHOLD_LOW);
            snprintf(line1, sizeof(line1), "DH:%.0f DL:%.0f C",
                    DIGITAL_THRESHOLD_HIGH,
                    DIGITAL_THRESHOLD_LOW);
        }
        s_lcd.showTwoLines(line0, line1);

        // Structured STDIO report (every 2 seconds)
        if ((displayCycle % REPORT_INTERVAL) == 0) {
            reportNumber++;
            char aTStr[8], dTStr[8];
            formatTemp(aTStr, sizeof(aTStr),
                        localSensor.analogTempC);
            formatTemp(dTStr, sizeof(dTStr),
                        localSensor.digitalTempC);

            printf("\r\n");
            printf("=== SENSOR REPORT #%lu ===\r\n",
                   (unsigned long)reportNumber);
            printf("Analog:  %s C (raw=%u, R=%.0f) [%s]\r\n",
                   aTStr, localSensor.analogRaw,
                   localSensor.analogResistance,
                   alertFullLabel(
                       localAlert.analogAlertState));
            printf("Digital: %s C [%s]\r\n",
                   dTStr,
                   alertFullLabel(
                       localAlert.digitalAlertState));
            printf("Alerts: A=%lu D=%lu Cycles=%lu\r\n",
                   (unsigned long)
                       localAlert.analogAlertCount,
                   (unsigned long)
                       localAlert.digitalAlertCount,
                   (unsigned long)
                       localAlert.conditioningCycles);
            printf("========================\r\n");
        }
    }
}
```

7.8. Application Layer: Lab 3.1 Entry Point

lab3_1_main.h

Listing 7.15 – lab3_1_main.h — Lab 3.1 Entry Point Interface

```
/**
```

```
 * @file lab3_1_main.h
 * @brief Lab 3.1 Entry Point Interface
 *
 * Declares setup and loop for the dual-sensor temperature
 * monitoring system with FreeRTOS preemptive scheduling.
 */

#ifndef LAB3_1_MAIN_H
#define LAB3_1_MAIN_H

void lab3_1Setup();
void lab3_1Loop();

#endif // LAB3_1_MAIN_H
```

lab3_1_main.cpp

Listing 7.16 – *lab3_1_main.cpp — Lab 3.1 Entry Point Implementation*

```
/**
 * @file lab3_1_main.cpp
 * @brief Lab 3.1 -- Dual-Sensor Temperature Monitoring
 *
 * Initializes STDIO, creates synchronization primitives, spawns
 * three FreeRTOS tasks, and starts the scheduler.
 */

#include "lab3_1_main.h"
#include "sensor_data.h"
#include "task_acquisition.h"
#include "task_conditioning.h"
#include "task_display.h"

#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <stdio.h>
#include "StdioSerial.h"

void lab3_1Setup() {
    stdioSerialInit(9600);

    printf("\r\n");
    printf("========================================\r\n");
    printf("  Lab 3.1 -- Dual-Sensor Temp Monitor\r\n");
    printf("  FreeRTOS | Tasks: 3 | Tick: %u Hz\r\n",
           (unsigned int)configTICK_RATE_HZ);
    printf("========================================\r\n");
    printf("Sensors: NTC(A0) + DS18B20(D2)\r\n");
```

```
    printf("Thresholds: HIGH=%.1f LOW=%.1f\r\n",
           ANALOG_THRESHOLD_HIGH, ANALOG_THRESHOLD_LOW);
    printf("Debounce: %u confirmations\r\n",
           (unsigned int)ALERT_DEBOUNCE_COUNT);
    printf("========================================\r\n\r\n");

    sensorDataInit();

    xTaskCreate(vTaskAcquisition, "Acquire",
        TASK_ACQUISITION_STACK, NULL,
        TASK_ACQUISITION_PRIORITY, NULL);

    xTaskCreate(vTaskConditioning, "Cond",
        TASK_CONDITIONING_STACK, NULL,
        TASK_CONDITIONING_PRIORITY, NULL);

    xTaskCreate(vTaskDisplay, "Display",
        TASK_DISPLAY_STACK, NULL,
        TASK_DISPLAY_PRIORITY, NULL);
}

void lab3_1Loop() {
    // All logic runs in FreeRTOS tasks.
}
```